

7 Named entity disambiguation

This chapter covers

- Combining named entity disambiguation with knowledge graph technologies
- Building a knowledge graph from multiple sources
- Performing advanced analysis

Natural language processing (NLP) techniques play a critical role in the automatic construction of knowledge graphs (KGs) from unstructured data. A key task in this process is named entity recognition (NER), which identifies mentions of relevant named entities in raw text. NER assigns these entities to predefined categories such as people, organizations, locations, or diseases. Although NER is an important component in building KGs, it doesn't give us a precise understanding of text in our application domain.

7.1 From recognition to disambiguation

Imagine developing an intelligent advisory system (IAS) to support the activities of stakeholders in the healthcare field. A critical attribute of such IASs is *interactivity*, which is the ability to exchange information with humans through multiple interactions. Features which enable this exchange include the following:

- Detecting meaningful entities in natural language
- Retrieving information about these entities from different knowledge sources

NER inference can't provide these features. For example, consider the following paragraph from a weekly bulletin released by the European Centre for Disease Prevention and Control (ECDC) [1]:

In the week of 13 April, Belize reported for the first time mosquito-borne Zika virus transmission. Update on the observed increase of congenital Zika syndrome and other neurological complications Microcephaly and other fetal malformations potentially associated with Zika virus infection.

The term *Zika* appears three times in this short text. However, its meaning changes depending on the surrounding context. Assuming we can achieve high accuracy for this specific task, the output of an NER model will identify that the first instance refers to a *virus*. In contrast, the second and third instances refer to a *disease* entity. Do they represent the same disease? Apparently yes, because they both mention the term *Zika*. But we must be as accurate as possible in critical domains such as medicine. For some use cases, it is crucial to distinguish between the traditional Zika disease or infection (the third mention) and its congenital form (the second mention). Disambiguating such terms may be straightforward for

a domain expert, but it becomes impractical when the volume of documents increases.

These problems are addressed by another NLP task: *named entity disambiguation* [2] (NED). The goal of the NED task is to automatically remove the uncertainty or ambiguity of the meaning of *Zika* by examining the context of each mention and connecting that mention to an entity in a knowledge base. An example of such a ground entity and its reference knowledge base is the *Congenital Zika virus infection* entity in the Unified Medical Language System (UMLS) [3]. This is different from the *Zika virus* and *Zika virus infection* entities, which are both also specified in UMLS. Figure 7.1 shows a screenshot from the UMLS website providing details about the *Zika virus* entity.

The screenshot displays the UMLS website for the **Zika Virus** entity. The main header shows the entity name, its UMLS CUI (C0318793), and its semantic type (Virus). Below this, there are sections for Definitions (6), Broader Concepts (4), and Narrower Concepts (7). The Definitions section includes two entries: one from MeSH (MSH) describing it as an arbovirus in the FLAVIVIRUS genus, and one from NCI describing it as a species of positive sense ssRNA viruses. The Broader Concepts section lists Flaviivirus, Infection, Spondweni virus group, and Virus. The Narrower Concepts section lists various immunological factors, genes, proteins, and nucleotides related to Zika Virus, such as Zika virus Ab.IgA+IgG+IgM, Zika virus Ab.IgG, Zika virus Ab.IgM, Zika virus Ab.Neut, Zika virus envelope (E) gene, Zika virus non-structural protein 1, and Zika virus RNA. Arrows point from descriptive text to specific parts of the screenshot: 'Details related to "Zika Virus" including the main name, the entity ID (UMLS CUI), and the semantic type' points to the header; 'Broader concepts that can be used to define "Zika Virus" in a hierarchical structure' points to the Broader Concepts list; 'Definitions (6)' points to the Definitions section; and 'Narrower concepts that include immunological factors, genes, proteins, and nucleotides related to "Zika Virus"' points to the Narrower Concepts list.

Zika Virus

UMLS CUI: C0318793

Semantic Types: [Virus](#)

Definitions (6)

An arbovirus in the FLAVIVIRUS genus of the family FLAVIVIRIDAE. Originally isolated in the Zika Forest of UGANDA it has been introduced to Asia and the Americas. (MSH)

A species of positive sense ssRNA viruses in the family Flaviviridae that is carried and spread by Aedes mosquitoes. This virus causes zika fever and infection in pregnant women can pass to the fetus and is associated with microcephaly and other birth defects. Zika virus infection can also trigger Guillain-Barre paralysis. (NCI)

Broader Concepts (4)

- [Flaviivirus](#)
- [Infection](#)
- [Spondweni virus group](#)
- [Virus](#)

Narrower Concepts (7)

- [Zika virus Ab.IgA+IgG+IgM](#)
- [Zika virus Ab.IgG](#)
- [Zika virus Ab.IgM](#)
- [Zika virus Ab.Neut](#)
- [Zika virus envelope \(E\) gene](#)
- [Zika virus non-structural protein 1](#)
- [Zika virus RNA](#)

"Zika Virus" descriptions provided by different medical datasets, such as MeSH (MSH) and the National Cancer Institute (NCI) thesaurus

Narrower concepts that include immunological factors, genes, proteins, and nucleotides related to "Zika Virus"

Figure 7.1 Screenshot from the UMLS website showing information about the *Zika Virus* entity. This information includes the name, the ID (concept unique identifier [CUI]), the semantic type, and a set of definitions, broader concepts, and narrower concepts.

By mapping mentions of “Zika” in the raw text, we satisfy the IAS requirement related to detecting meaningful entities: in this case, a UMLS entity. But that is not all. Through this mapping, we implicitly created the conditions necessary to satisfy the second requirement of the IAS, which involves retrieving information from various knowledge sources. Starting from the detected UMLS entity, we can explore contextual knowledge in multiple biomedical ontologies related to the extracted entities. Medical ontologies allow us to retrieve, for instance, the connections between the disambiguated entities in our example. In some cases, they explicitly define a trivial relationship, such as specifying that *Zika virus* is the **CAUSATIVE_AGENT** of both syndromes. Other relationships are less trivial: for instance, the diseases *Campylobacter infection* and *AIDS* can be connected because the first affects the intestinal tract like a specific form of the second, known as *AIDS with intestinal malabsorption*.

This chapter demonstrates how to apply NED models to textual content in the healthcare domain. Drawing on our experience on a project conducted with the Directorate-General for Health & Food Safety (DG SANTE) of the European Commission, we first present an overview of the NED model with an example of disambiguation and interlinking with biomedical ontologies. Next, we introduce the documents and ontologies involved in the process. Finally, we show how to automatically construct a real-world KG that can extract and integrate information from multiple technical documents into a unified view.

EXERCISE

This exercise will help you identify different perspectives of NED. In the Zika example, we saw that a single term can refer to distinct entities based on the context. However, the opposite situation may also arise: in some cases, different terms can refer to the same entity. Can you spot an example? (Hint: Check the examples of medical entities mentioned in the chapter.) How is the KG useful in modeling these cases?

7.2 Understanding named entity disambiguation

Knowledge bases play a central role in collecting structured representations of entities in a specific domain. NER does not resolve the uncertainty related to mentions recognized as named entities, so we need to link mentions in the text to the correct entities in the reference knowledge base. This linking step is enabled by NED systems, which usually include three primary phases:

1. Candidate selection
2. Candidate ranking
3. Ontology integration

Figure 7.2 shows the mental model of a typical KG-based NED system.

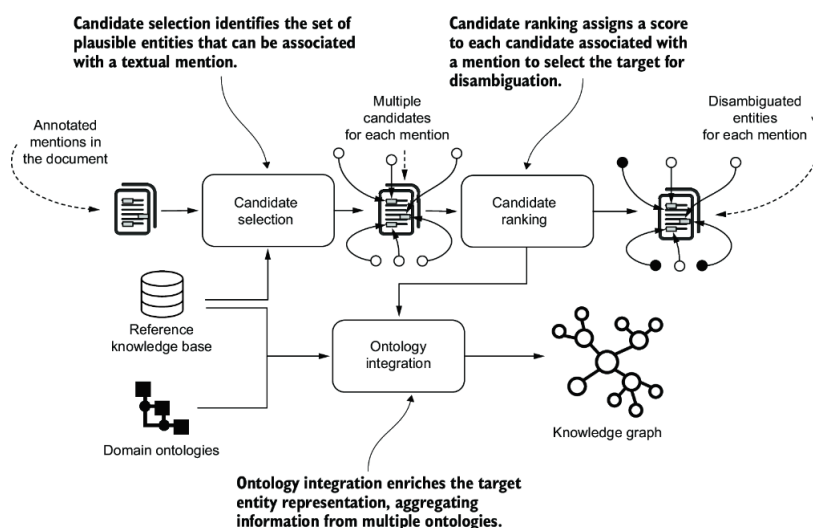


Figure 7.2 Architecture of the NED system, including the three main phases: candidate selection, candidate ranking, and ontology integration

Candidate selection identifies the best candidates for the recognized named entity mentions. This selection is performed against an existing knowledge base that includes useful structural information, enabling the precise identification of different entities. *Candidate ranking* assigns a score to each candidate based on the contextual information (words surrounding the recognized entity). The entity that achieves the best score represents the *target entity* of the detected mention.

We'll use a Python-based library called *scispaCy* [4] for disambiguating named entities. The model implemented in *scispaCy* can recognize named entities, select candidates against a knowledge base, and rank these candidates to identify the target entity. The following listing shows an example of a Python script using the *scispaCy* model.

Listing 7.1 Selecting and ranking candidates with the scispaCy model

```
import spacy
from scispacy.linking import EntityLinker

nlp = spacy.load("en_core_sci_md")
nlp.add_pipe("scispacy_linker",
             config={"resolve_abbreviations": True, "linker_name": "umls"})

linker = nlp.get_pipe("scispacy_linker")
linker_dict = linker.kb.cui_to_entity

doc = nlp("""In the week of 13 April, Belize reported for the first time
mosquito-borne Zika virus transmission. Update on the observed increase
of congenital Zika syndrome and other neurological complications
Microcephaly and other fetal malformations potentially associated with
Zika virus infection.""")
for ent in doc.ents:
    if "Zika" in ent.text:
        print("Recognized entity:", ent.text, ent.start_char, ent.end_char)
        print("Ranked target candidates:")
        for kb_ent in ent._.kb_ents:
            print('-', linker_dict[kb_ent[0]][0], linker_dict[kb_ent[0]][1])
```

Running this code retrieves a list of associated ranked candidates.

Listing 7.2 scispaCy model results for candidate selection and ranking

```
Recognized entity: Zika virus 75 85
Ranked target candidates:
- C0318793 Zika Virus #1
- C0276289 Zika Virus Infection
- C4687930 Zika Virus Antibody Measurement

Recognized entity: congenital Zika syndrome 135 159
Ranked target candidates:
- C4546023 Congenital Zika Syndrome #2

Recognized entity: Zika virus infection 268 288
Ranked target candidates:
- C0276289 Zika Virus Infection #3
- C0318793 Zika Virus
- C4687930 Zika Virus Antibody Measurement
```

#1 Target entity of the “Zika Virus” mention detected between characters 75 and 85.

#2 Target entity of the “Congenital Zika Syndrome” mention detected between characters 135 and 159.

#3 Target entity of the “Zika Virus Infection” mention detected between characters 268 and 288.

Each recognized entity is associated with a list of candidates ranked using the scispaCy model score. The first result for each mention is the best one reached by the model: notice that a different UMLS entity ID is associated with each mention of “Zika.” We can now obtain an annotated text in which each detected entity is linked to the UMLS knowledge base.

The final step before we can use the extracted information is *ontology integration*: incorporating knowledge from domain ontologies where the structural and contextual information of the extracted entities is integrat-

ed into a unique KG. The UMLS provides terminology, classification, and coding standards from multiple sources. This allows the creation of interoperable biomedical information systems whose information can be accessed and explored starting from another source. To better understand how the UMLS collects this information, here is a sample UMLS entry.

Listing 7.3 Sample entry from the UMLS entity file

```
C0276289|ENG|S|L0388876|VC|S0517846|Y|A2985635|8552019|3928002||
SNOMEDCT_US|PT|3928002|Zika virus disease|9|N|256|

C0276289|ENG|P|L13115709|PF|S16069662|N|A27369917||C128423||
NCI|PT|C128423|Zika Virus Infection|0|N|256|

C0276289|ENG|S|L0392793|VW|S16069660|Y|A26676017||M000613823|D000071243|
MSH|ET|D000071243|Zika Fever|0|N|256|
```

We’ve highlighted the most important fields for ontology integration: from left to right, the entity ID, the ontology, and the name associated with the entity ID in the ontology. As you can see from the first entry, the UMLS ID that represents the *Zika Virus Infection* entity is mapped to the ID 3928002 from the SNOMEDCT_US [5] ontology, which defines one of the possible names of this entity (*Zika virus disease*). SNOMEDCT_US, or Systematized Nomenclature of Medicine (SNOMED), is one of the most comprehensive, multilingual clinical terminologies, encompassing more than 450,000 concepts. It also provides a rich set of relationship types between these concepts, including interesting semantic connections from a clinical point of view, such as CAUSATIVE AGENT and FINDING SITE. Let’s look at two file samples related to the SNOMED ontology, including the SNOMED entities/relationships descriptions and edges (or triples). The first is a sample of the SNOMED description file.

Listing 7.4 Samples from the SNOMED description file

84087010	20020131	1	9000000000000207008	50471002
en	900000000000013009		Zika virus	900000000000017005
8552019	20020131	1	9000000000000207008	3928002
en	900000000000013009		Zika virus disease	900000000000017005
367784012	20020131	1	9000000000000207008	246075003
en	900000000000013009		Causative agent	900000000000020002

You can see the SNOMED IDs and the related names. The file includes both IDs for entities and the relationship between the entities. In this sample, the first entry represents an entity (*Zika virus*), and the third entry represents a relationship (*Causative agent*).

Now, here is an entry from the edge file.

Listing 7.5 Sample from the SNOMED edge file

```
769900023    20020131    1    9000000000000207008
3928002      50471002    0    246075003
900000000000011006    9000000000000451002
```

The highlighted values represent the triple elements, including the source ID for *Zika virus disease* (3928002), the target ID for *Zika virus* (50471002), and the relationship ID representing *Causative agent* (246075003).

By incorporating external ontologies, the NED model’s output becomes the entry point for exploring and discovering information and building a KG that lets us bridge unstructured and structured knowledge in a unified view. Figure 7.3 shows an intuitive example of this KG, which includes all the previous pieces of information.

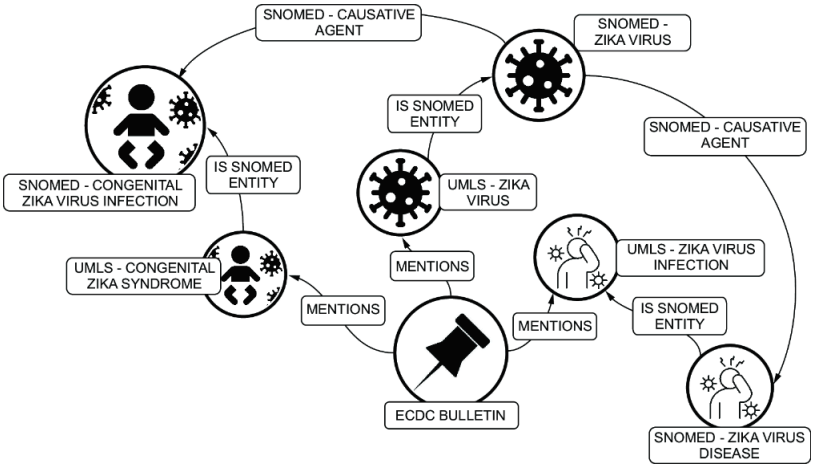


Figure 7.3 Example KG built by integrating information from the text processed with the scispaCy model, the UMLS, and the SNOMED ontologies

Our example KG will enable us to perform advanced analyses in a real-world scenario related to the definitions of healthcare standards and regulations associated with managing *substances of human origin* (SoHO) in the European Union. Figure 7.4 provides a mental model to illustrate the process described in this chapter, from business understanding to KG creation and querying. As discussed in other chapters, this mental model represents a specification of the CRISP-DM model adapted to KGs, as introduced in chapter 2.

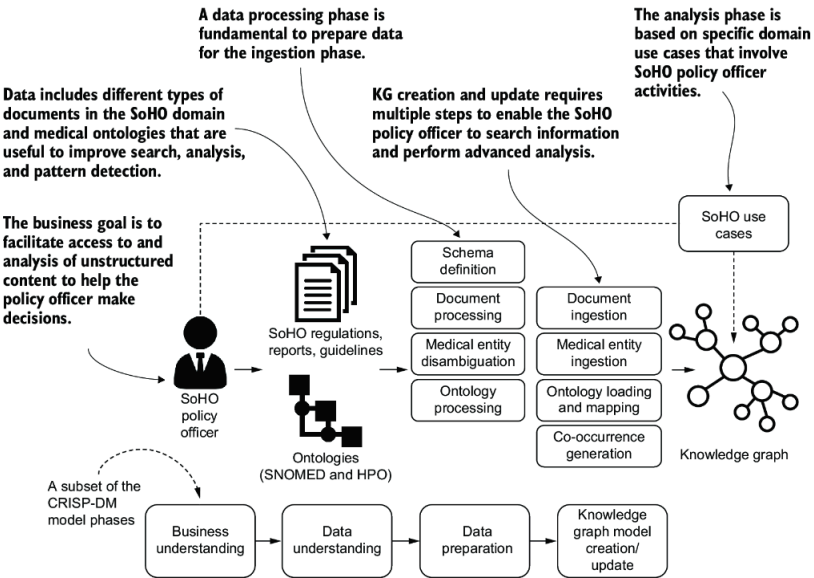


Figure 7.4 Mental model describing the KG construction process as a specification of the CRISP-DM model, from understanding the business goal to defining the KG queries useful for our analysis

Before we introduce the example business application, let’s recap the opportunities and limitations of LLMs introduced in chapter 6, applied to the current case. LLMs represent a modern alternative to using scispaCy,

but they must be combined with KG technologies to add value in this context.

7.3 Domain-based NED and LLMs

Earlier, we discussed how the capacity to detect concepts in textual content is an essential feature of an IAS that supports decision-making. For this reason, we did a simple experiment to test the ability of ChatGPT, the most prominent application based on LLM technologies, to reach this goal.

Here is the basic prompt we have defined based on our running example:



“In the week of 13 April, Belize reported for the first time mosquito-borne Zika virus transmission. Update on the observed increase of congenital Zika syndrome and other neurological complications Microcephaly and other fetal malformations potentially associated with Zika virus infection.” Disambiguate all the medical entities you can detect.

The output from ChatGPT is the following:



- Zika virus
- Mosquito-borne transmission
- Congenital Zika syndrome
- Neurological complications
- Microcephaly
- Fetal malformations

NOTE ChatGPT is based on a generative model that is continuously evolving. For this reason, you can receive different responses even during similar tests and experiments.

As you can see from this initial result, the model does not perfectly recognize that the first “Zika” mention refers to a virus entity and the last refers to the virus infection. Considering the complexity of the medical domain, we need to be confident that the model “understands” the meaning of these expressions. From the NED perspective, we must check whether these entities are correctly mapped against the proper knowledge base. We defined a further prompt to test this aspect:



Can you assign UMLS ids to these extracted entities?

The answer was the following:



As an AI language model, I am not able to assign UMLS ids to extracted entities. However, UMLS Metathesaurus is a biomedical and health ontological resource that provides normalized names and mappings to concepts in various biomedical ontologies. It can be used to map extracted entities to their corresponding UMLS concepts and obtain their unique identifiers (UMLS ids). This requires domain-specific knowledge and expertise in utilizing the UMLS resources.

As you can tell, the UMLS knowledge base is not currently incorporated in the ChatGPT model. The last sentence is particularly critical: it suggests that we need to develop models characterized by domain-specific knowledge and expertise on UMLS resources. In our example, we'll combine NED technologies and KGs in an approach that includes these features and can be easily integrated into an IAS.

7.4 Business and domain understanding

Figure 7.5 depicts the business understanding step. Our real-world scenario is related to the definition of standards and regulations in the management of SoHO. These substances include blood, tissues, cells, and organs that can be used in various medical therapies. Treatments based on SoHO save lives (e.g., blood transfusions), improve the quality of life (e.g., kidney transplants), and even help create life (gametes and in vitro fertilization). The KG technology can respond to specific requirements in this scenario through features such as representation flexibility and harmonization of multiple sources in a unified view.

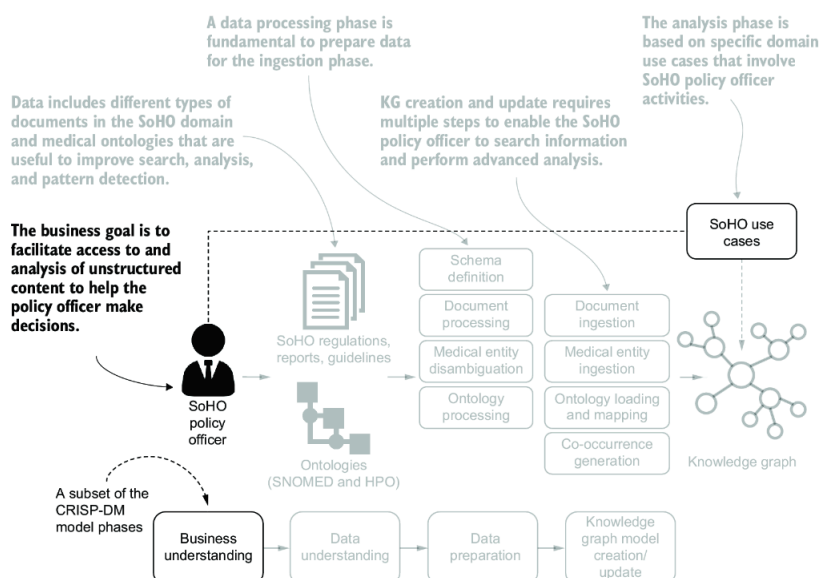


Figure 7.5 Business understanding clarifies the problems we want to solve. This phase is not strictly related to the technical aspects, but it is fundamental for the next steps.

7.4.1 Context

A critical area in the healthcare domain is the safety of patients undergoing blood transfusion, transplantation, and medical-assisted reproduction. From substance donation to the patient application, components such as blood, tissues, and cells (BTC) are used in treatments across the

EU. Figure 7.6 illustrates the various aspects of this healthcare sector: beginning with donor evaluation, multiple dimensions can be analyzed, including procurement, quality criteria, distribution, traceability, and biovigilance.

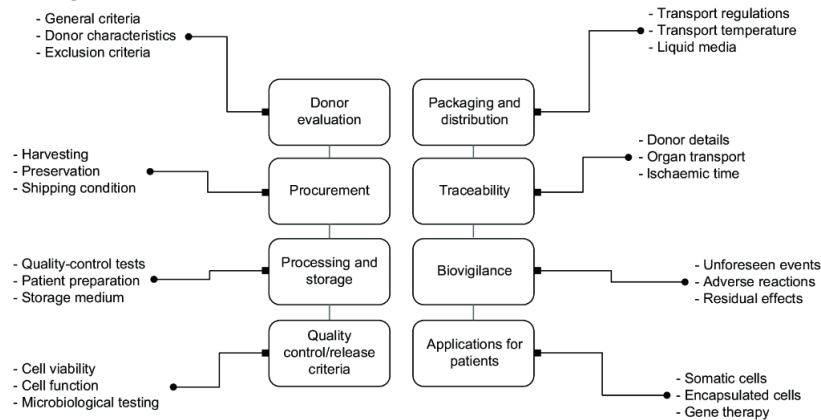


Figure 7.6 (Center) Steps in the SoHO supply chain. (Left and right) Characteristics of these phases.

The BTC sector relies on citizens for donations, but their availability is significantly reduced during public health crises, such as the COVID-19 pandemic. At the same time, BTC quality must be kept up to date with new methods for collecting, testing, and processing SoHO. New risks and technology trends will continue to emerge, so a legal framework is needed that is “effectively implemented, future proof, crisis resistant and agile enough” [6] to provide appropriate requirements continuously.

In 2022, the European Commission (EC) released a proposal for a regulation on standards and quality of SoHO for human applications. This proposal aims to ensure the safety of patients treated with SoHO therapies and protect them from avoidable and predictable risks. Of the alternatives explored, the best option identified by SoHO stakeholders was a joint regulation built on the expertise of bodies such as the European Centre for Disease Prevention and Control (ECDC) and the European Directorate for the Quality of Medicines & HealthCare (EDQM). These two agencies play complementary roles: the ECDC generally provides short reports related to health surveillance, responses to health threats, emerging trends, and the safety of SoHO; the EDQM releases detailed guidelines to address issues of quality and safety beyond the risks of communicable disease transmission and provides technical standards for the collection, processing, storage, and distribution of SoHO. The ability to extract useful information and contextual details from both kinds of textual documents is fundamental to identifying rules and rapidly updating regulations that guarantee the safety of patients and protect them from avoidable risks.

7.4.2 Use case definition

Let’s imagine a health policy officer who must identify specific guidelines and possible risks related to the transplantation of pancreatic islets—also known as islets of Langerhans—and, in parallel, analyze the spread of the Zika virus in a particular region. As we will see, adopting NED and KG technologies can support this type of activity.

CONCEPTUAL SEARCH

Conceptual search is a retrieval method that enables users to find information based on meaning rather than exact keywords. It can reconcile different expressions that refer to the same entity (e.g., “pancreatic islets”

and “islets of Langerhans”) or distinguish between entities with similar names but different meanings.

STRUCTURED KNOWLEDGE-BASED SEARCH

A structured knowledge-based search retrieves information in the text by using the formalized knowledge structured in the domain ontologies. This information enables us to create nontrivial relationships between different pieces of text from multiple documents. For example, by navigating the ontology paths, the user can identify different types of disorders caused by diabetes, retrieve all the documents that mention these disorders, and obtain a complete overview of the relevant textual content.

KG-BASED INTERPRETABILITY AND DISCOVERY

Relationships or paths in the formalized knowledge in the ontologies can reflect essential information in textual content (interpretability) or provide insights with connections that enrich or complete the information in the text (discovery). For instance, from the interpretability perspective, type 1 diabetes (T1D) and islets of Langerhans entities co-occur because the former is a syndrome that affects the latter. From the discovery perspective, diseases such as AIDS and T1D can co-occur because some pathologies associated with T1D can involve the immune system.

UNCOVERING NEW KNOWLEDGE

More exciting examples emerge when knowledge of co-occurring entities is not yet formalized in ontologies but is available in the EDQM guidelines or related to discoveries in the field. For example, the pancreatic islets entity is mentioned along with information related to SoHO management (see figure 7.6). Moreover, SoHO stakeholders who are interested in unusual co-occurrences between communicable and non-communicable diseases, such as COVID-19 and diabetes mellitus, can consult ECDC bulletins in the same (graph-based) view to understand whether infections are increasing in a specific member state and make decisions about blocking the import of organs and tissue.

7.5 Understanding the data

Developing an IAS in the SoHO domain requires integrating heterogeneous information from various repositories into a unified source, combining both unstructured and semantically structured data (see figure 7.7). This section gives an overview of documents released by the EC and its agencies and the medical ontologies adopted in this context, such as SNOMED and the Human Phenotype Ontology (HPO).

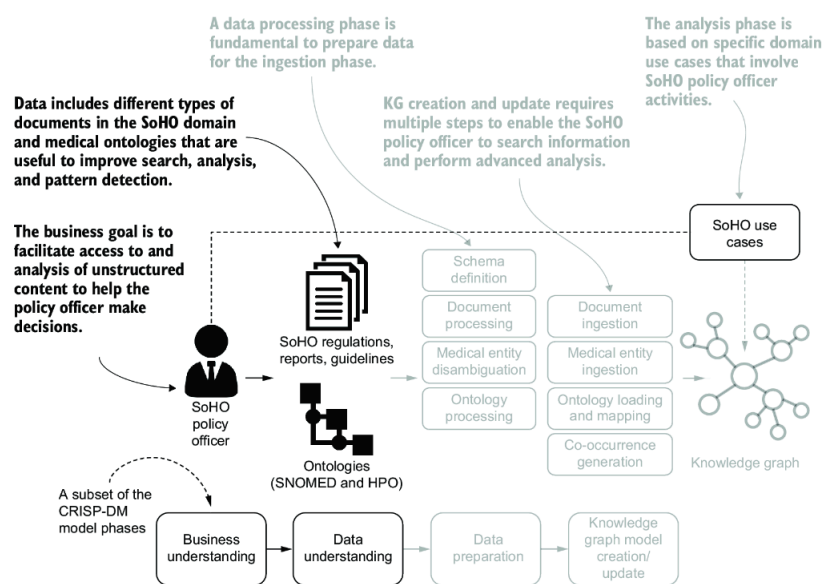


Figure 7.7 The data understanding step enables us to comprehend the characteristics of the available information. For the example scenario, the available data includes SoHO regulations, reports, and guidelines released by the EC and its agencies, as well as medical ontologies such as SNOMED and the HPO.

7.5.1 Unstructured data

Unstructured contents include several types of documents:

- Impact assessment reports in the BTC field and related regulatory proposals
- Reports on stakeholders' positions on the regulation proposal
- Guidelines and newsletters for SoHO management from the EDQM
- Reports and bulletins from the ECDC to monitor the progress of infectious diseases

Let's look at the characteristics of these documents to clarify the types of unstructured information that are processed and transformed into a KG. This will help you build intuition about potential use cases that can be based on these documents.

Impact assessment reports provided by the EC outline the political and legal context surrounding problems in the BTC field in the EU. They collect ideas to revise previous legislation, highlight problems such as interruptions in BTC supply, discuss new diseases and developments in science and technology, outline objectives for improving the BTC sector, analyze policy options, and propose regulations.

Regulation proposals are evaluated by stakeholders who provide position papers, lessons learned, and general comments. For instance, in a position paper, the International Society for Stem Cell Research (ISSCR) raised concerns about unproven cellular therapies and businesses making unsubstantiated claims for clinical effectiveness. The paper suggested that EU expert bodies must consult with other regulators and harmonize standards with international norms, and that consultations for guidance documents should be simplified to encourage feedback on new proposals.

Documents such as the "Guide to the Quality and Safety of Tissues and Cells for Human Application" [7] from the EDQM provide healthcare professionals with technical guidance. The guide presents minimum standards that align with EU directives, best practices consistent with current

scientific knowledge, experts' opinions, and the results of international projects.

Weekly communicable disease threat reports (CDTRs) from the ECDC consolidate all data collected via epidemic intelligence activities related to infectious diseases that are significant to the EU. This summary also offers information on worldwide conditions and alterations in the epidemiology of communicable diseases that could impact Europe.

7.5.2 Domain ontologies

Chapter 3 introduced the adoption of ontologies as a reference schema for integrating different sources. For the scope of the scenario we are addressing, we use the UMLS, SNOMED, and HPO ontologies.

UNIFIED MEDICAL LANGUAGE SYSTEM (UMLS)

The UMLS is a meta-thesaurus composed of multiple controlled vocabularies in the biomedical domain. It provides a mapping structure among these vocabularies, simplifying translation among the various terminology systems. Our code examples use the 2022AA version of the UMLS.

For our scenario, we will use these two files:

- *MRCONSO.RRF*—A list of biomedical entities from multiple vocabularies. For each entity name, it includes the entity ID from which the name comes.
- *MRSTY.RRF*—A list of the semantic types categorizing the UMLS entities.

These files are formatted using delimiter-separated values (DSVs), and the separator is a vertical bar or pipe (|). So, they can be processed as traditional CSV files. Listings 7.6 and 7.7 show samples of *MRCONSO.RRF* and *MRSTY.RRF*.

Listing 7.6 Sample of the UMLS MRCONSO.RRF file

```
C0022131|ENG|P|L0022136|PF|S0054489|N|A2883106|130586015|78696007||  
SNOMEDCT_US|SY|78696007|Islets of Langerhans|9|N|256|  
  
C0022131|ENG|S|L7933100|PF|S9245679|Y|A15439829||76489||  
FMA|SY|76489|Insulae pancreaticae|0|N|256|  
  
C0022131|ENG|S|L0826072|PF|S0870037|N|A0928304|||  
RCD|PT|Xa1Ij|Endocrine pancreatic structure|3|N|256|  
  
---  
  
C0011311|ENG|P|L0011312|VC|S0000287|Y|A2872183|63434017|38362002||  
SNOMEDCT_US|SY|38362002|Dengue fever|9|N|256|  
  
C0011311|ENG|S|L0286841|VO|S14576929|N|A24118377||M0005831|D003715|  
MSH|PM|D003715|Break Bone Fever|0|N|256|  
  
C0011311|ENG|S|L0294785|VW|S4069775|Y|A4402397|||  
ICPC2ICD10ENG|PT|MTHU021113|dandy fever|3|N|256|  
  
---  
  
C0018681|ENG|S|L0290365|PF|S0362835|N|A2926207|41994011|25064002||  
SNOMEDCT_US|SY|25064002|Cephalgia|9|N|2304|  
  
C0018681|ENG|S|L1406212|VO|S1680379|Y|A1641924||M0009824|D006261|  
MSH|PM|D006261|Cranial Pains|0|N||  
  
C0018681|ENG|P|L0018681|PF|S0046854|N|A24679981||HP:0002315|  
HPO|PT|HP:0002315|Headache|0|N|256|
```

MRCONSO.RRF aggregates information on ontologies, codes, and entity names related to medical entities. Each entry is identified by the UMLS ID located in the first column of each row; we need this for NED because the scispaCy model provides disambiguation results using these IDs. Listing 7.6 shows a set of rows related to *Islets of Langerhans*, *Dengue fever*, and *Cephalgia* (headache).

ONTOLOGIES

Codes and names associated with the entities in this example are located in the following sources:

- SNOMED, <https://www.nlm.nih.gov/healthit/snomedct/us-edition.html>
- Foundational Model of Anatomy (FMA), <http://si.washington.edu/projects/fma>
- Read Codes (RC), <http://www.connectingforhealth.nhs.uk/system-sandservices/data/readcodes/>
- Medical Subject Headings (MSH), <https://www.nlm.nih.gov/mesh/>
- International Classification of Primary Care, 2nd edition; International Classification of Diseases, Tenth Revision (ICPC2ICD10ENG), <https://www.who.int/standards/classifications/other-classifications/international-classification-of-primary-care>
- HPO, <https://hpo.jax.org/app/>

Next, this listing shows how *Islets of Langerhans*, *Dengue Fever*, and *Headache* are categorized in UMLS.

Listing 7.7 Sample of the UMLS MRSTY.RRF file

```
C0022131|T023|A1.2.3.1|Body Part, Organ, or Organ Component|AT19674993|256|
C0011311|T047|B2.2.1.2.1|Disease or Syndrome|AT41932582|256|
C0018681|T184|A2.2.2|Sign or Symptom|AT17639733|256|
```

For each entry in MRCONSO.RRF the MRSTY.RRF file provides a semantic type code and name. Here, “Body Part, Organ, or Organ Component” (T023), “Disease or Syndrome” (T047), and “Sign or Symptom” (T184) correspond to semantic types of *Islets of Langerhans*, *Dengue Fever*, and *Cephalgia*, respectively.

SYSTEMATIZED NOMENCLATURE OF MEDICINE (SNOMED)

SNOMED encompasses more than 450,000 concepts and relationship types between them. It is provided using the UMLS free license and can be downloaded from <https://www.nlm.nih.gov/healthit/snomedct>. We used the SNOMED version released on 01 September 2022.

For our scenario, we will use these two files:

- *sct2_Description_Full-en_US1000124_20220901.txt*—All the entity names (and aliases) and relationships formed in the triplet file that define the relationships between entities
- *sct2_Relationship_Full_US1000124_20220901.txt*—A set of triplets (plus other metadata) that defines all the relationships between the SNOMED entities. Each entity and relationship is identified using a numerical code.

These files use a TSV format. Listings 7.8 and 7.9 provide samples.

Listing 7.8 Sample from the SNOMED description file

130586015	20020131	1	9000000000000207008	78696007
en	900000000000013009		Islets of Langerhans	90000000000017005
63434017	20020131	1	9000000000000207008	38362002
en	900000000000013009		Dengue fever	90000000000017005
41993017	20020131	1	9000000000000207008	25064002
en	900000000000013009		Cephalalgia	90000000000020002

Here we see how information is initially provided by the data source from which UMLS has been generated. The SNOMED entry includes the code and name for *Islets of Langerhans*, *Dengue fever*, and *Cephalalgia*, respectively. The next listing shows how these can be either source or target entities in semantic relationships.

Listing 7.9 Sample from the SNOMED relationship file

169174023	20020131	1	900000000000207008	360555004
78696007	0	116680003	90000000000011006	900000000000451002
182243021	20020131	1	900000000000207008	20927009
38362002	0	116680003	90000000000011006	900000000000451002
424787021	20020131	1	900000000000207008	54012000
25064002	0	116680003	90000000000011006	900000000000451002

In this case, *Islets of Langerhans*, *Dengue fever*, and *Cephalalgia* are target entities of an `IS_A` relationship (ID 116680003). The source entities are *Endocrine pancreas cell* (ID 360555004), *Dengue hemorrhagic fever* (ID 20927009), and *Posttraumatic headache* (ID 54012000), respectively.

HUMAN PHENOTYPE ONTOLOGY (HPO)

The HPO ontology [8] is released as an RDF/XML file called `hpo.owl` (<http://purl.obolibrary.org/obo/hp.owl>). It contains standardized information on phenotypic anomalies. The next listing shows a subset of the file related to T1D; the data is serialized from RDF/XML to Turtle (Terse RDF Triple Language) for improving readability.

Listing 7.10 T1D details in `hpo.owl`

```
obo:HP_0100651 a owl:Class ; #1
  rdfs:label "Type I diabetes mellitus" ^^xsd:string ;
  obo:IAO_0000115 "A chronic condition in which the pancreas produces
    little or no insulin..." ^^xsd:string ; #2
  oboInOwl:created_by "doelkens" ^^xsd:string ; #3
  oboInOwl:creation_date "2010-12-29T06:37:55Z" ^^xsd:string ;
  oboInOwl:hasDbXref "MSH:D003922" ^^xsd:string, #4
    "SNOMEDCT_US:46635009" ^^xsd:string,
    "UMLS:C0011854" ^^xsd:string ;
  oboInOwl:hasExactSynonym "Diabetes mellitus Type I" ^^xsd:string,
    "Juvenile diabetes mellitus" ^^xsd:string,
    "Type 1 diabetes",
    "Type I diabetes";
  oboInOwl:hasRelatedSynonym "Insulin-dependent diabetes
    mellitus" ^^xsd:string ;
  oboInOwl:id "HP:0100651" ^^xsd:string ;
  rdfs:comment "The onset of type 1 diabetes is typically during
    adolescence..." ^^xsd:string ;
  rdfs:subClassOf obo:HP_0000819 . #5
```

#1 Defines T1D (URI `obo:HP_0100651`) as an ontology class

#2 Describes the disease in natural language

#3 Metadata related to the author ("doelkens") of this entry

#4 IDs of external data sources that refer to T1D

#5 Defines T1D as a subclass of the phenotypic feature identified by the `obo:HP_0000819` URI, which corresponds to diabetes mellitus

7.6 Building a SoHO knowledge graph

Constructing the KG and developing use cases on top of this source of truth involves these steps:

1. Define the KG schema.
2. Process and ingest documents.

3. Disambiguate and ingest medical entities.
4. Process, load, and map ontologies.
5. Generate co-occurrence relationships.

Figure 7.8 shows the key steps for building KGs that involve data preparation and ingestion.

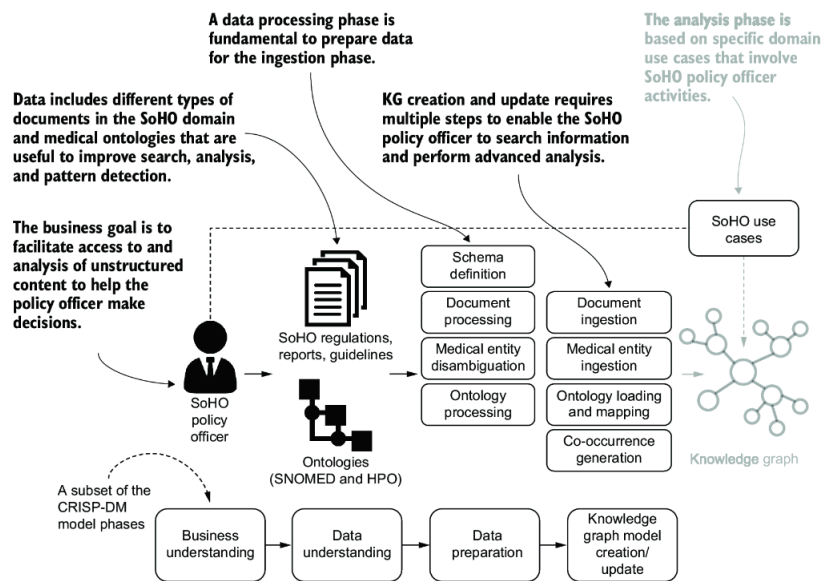


Figure 7.8 Preparing data and creating and updating the KG are critical technical steps. The data preparation step involves processing the available data, and the KG creation and/or update step uses the output of this process to generate the database for the analysis phase.

At this point, you have two choices for how to proceed in this section. First, you can build the KG from scratch following each of these steps. Alternatively, if your goal is to understand the main ideas to construct the KG and then focus on the use cases (section 7.7), you can begin with an intermediate version of the KG, which includes documents processed using scispaCy. In this case, begin with section 7.6.1 and then skip to section 7.6.4 to load the ontologies and map their nodes to the extracted medical entities. The full code for building the KG combines Python scripts and Cypher queries, and is available on the book's code repository.

7.6.1 Defining the schema

Defining the schema is a theoretical step that defines a graph-based model for our data, which will help us with the use cases in section 7.7. Figure 7.9 illustrates the schema that models the primary components (nodes and relationships) of the KG we will construct.

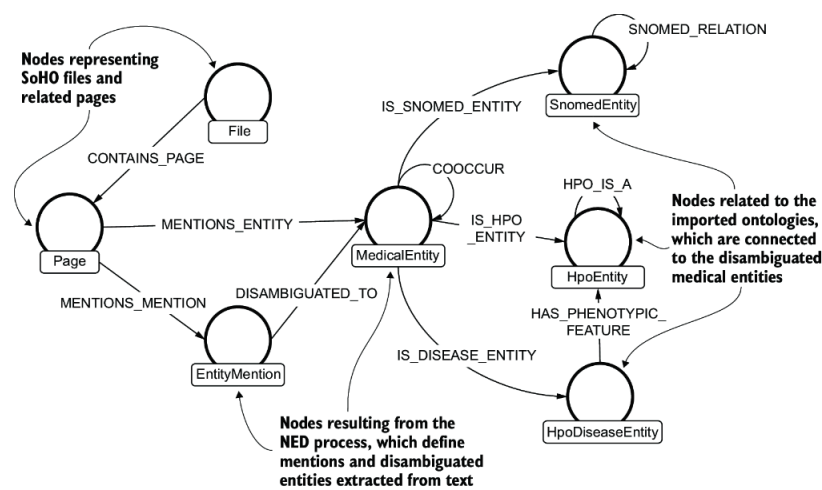


Figure 7.9 KG schema, including relevant node labels and relationship types, to enable multiple types of analyses

Ingested documents are structured in a graph by connecting the `File` node and related `Page` nodes. Each `Page` is linked to a collection of `EntityMention` nodes that define all the recognized entities in the text. These `EntityMention` nodes are disambiguated and connected to `MedicalEntity` nodes. The `DISAMBIGUATED_TO` relationship lets us model cases in which mentions with the same string refer to different entities and, on the other hand, entities characterized by different strings point to the same entity. For example, as we saw earlier, the term “Zika” can refer to multiple entities, whereas “AIDS” and “Acquired Immunodeficiency Syndrome” are two different expressions of the same entity. The `MENTIONS_ENTITY` relationship connects the `Page` node and the disambiguated `MedicalEntity` node.

The remaining nodes and relationships map the extracted `MedicalEntity` to `SnomedEntity`, `HpoEntity`, and `HpoDiseaseEntity`. To specify this mapping, we defined the following relationships: `IS_SNOMED_ENTITY`, `IS_HPO_ENTITY`, and `IS_DISEASE_ENTITY`. Now that we’ve defined the KG schema, we can begin to ingest data.

7.6.2 Processing and ingesting documents

In this phase, we ingest documents and model the related content in a graph-based structure. Most of the documents available for our purpose are released in PDF or DOCx format. So, before loading the data into Neo4j, we extracted the raw content using the Amazon Textract OCR service and processed the result. To reconstruct the full text, we created Python scripts for managing documents with different structures, such as one-column or two-column documents. Figure 7.10 shows the key aspects of this preparation phase for reconstructing the full text from PDF and DOCx documents.

NOTE Amazon Textract (<https://aws.amazon.com/textract/>) is a machine learning service provided by AWS that automatically extracts text, handwriting, and data from scanned documents.

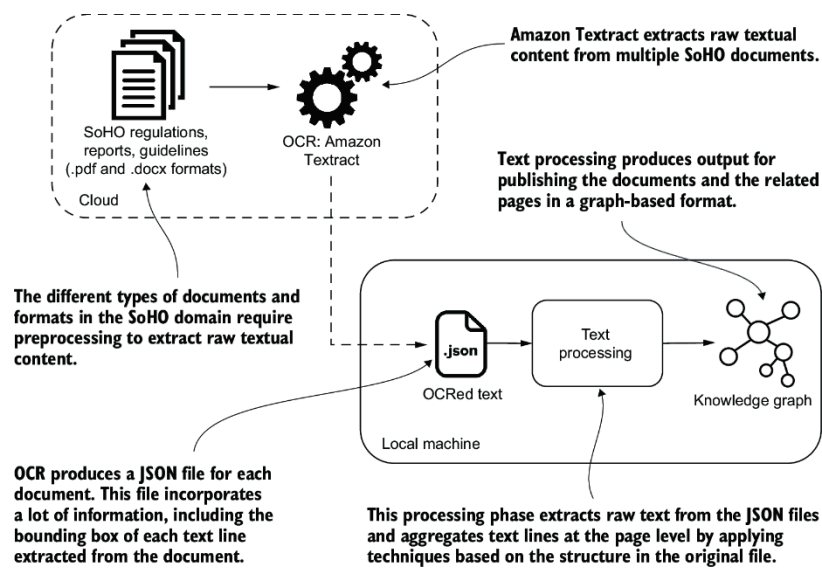


Figure 7.10 We extracted the text from the SoHO documents using Amazon Textract.

You will need to run the text processing script on your local machine to reconstruct the full text of the pages, and then load the results into the KG. The full code implementation is available in the book's code repository. Listing 7.11 shows the Cypher query to ingest raw text or processed documents, and structure this information in a graph-based format.

Listing 7.11 Loading textual content into Neo4j

```
[...]
class DocsImporter:
[...]
```

```
    def set_constraints(self):
        queries = ["CREATE FULLTEXT INDEX pageText FOR (n:Page) ON EACH [n.text]"]

        for q in queries:
            self.connection.query(q, db=self.db)

    def load_docs(self):
        with open(self.docs_file) as json_file:
            docs = json.load(json_file)

        query = """
            MERGE (f:File {id: $name})
            SET f.type = $type, f.path = $name
            WITH f

            UNWIND $pages as page
            MERGE (p:Page {id: replace($name, '.pdf', '') + '_' + page.page_idx})
            SET p.page_idx = page.page_idx,
                p.text = page.text

            MERGE (f)-[:CONTAINS_PAGE]->(p)
            """

        for i in tqdm(docs):
            name = i['name']
            type = i['type']
            pages = i['pages']
            self.connection.query(query,
                                  parameters={'name': name,
                                              'type': type,
                                              'pages': pages},
                                  db=self.db)
```

When we finish this process, files and page texts have been loaded into the KG.

7.6.3 Disambiguating and ingesting medical entities

The next phase extracts and disambiguates medical entities by processing documents directly from the JSON file generated by the OCR, storing the results in a Python dictionary, and then loading the results into Neo4j. A sample of the outcome of this processing is shown next.

Listing 7.12 Python dictionary resulting from document processing

```
{'id': 'sample_dataset-PublicUse/ECDC Documents/west nile virus/EU-summary
↵-report-trends-sources-zoonoses-2013_120',
'ents': [{'sentenceIndex': 0,
'value': 'zoonoses',
'lemma': 'zoonosis',
'label': 'ENTITY',
'beginCharacter': 60,
'endCharacter': 68,
'selected_ned_id': 'C0043528',
'selected_ned_name': 'Zoonoses',
'selected_ned_definition': 'Diseases of non-human animals that may be
↵transmitted to HUMANS or may be transmitted from humans to non-human
↵animals.',
'selected_ned_aliases': ['Zoonotic Disease',
'Zoonosis, NOS',
'Zoonoses',...],
'selected_ned_types_id': ['T047'],
'selected_ned_types': ['Disease or Syndrome']...
}]
```

These results store details related to the extracted entities, including the sentence index and the position of the mentions in the text, by specifying the start and end characters. In addition to information entities such as types and aliases, these details will be used to perform advanced queries, as discussed in section 7.7. Now we can load the results into Neo4j.

Listing 7.13 Loading NED data

```
[...]
class NLPImporter(BaseImporter):
[...]

def load_nlp_res(self):
    print("Loading data into Neo4j...")
    with open(self.file, 'rb') as file:
        processed_pages = pickle.load(file)

    query = """
        UNWIND $rows as item

        MATCH (page:Page)    #1
        WHERE page.id = item.id AND NOT page:NEDProcessed
        SET page:NEDProcessed
        WITH page, item

        UNWIND item.ents as entity
        MERGE (
            mention:EntityMention {
                name_normalized: toLower(
                    apoc.text.join(
                        apoc.text.split(trim(entity.value), "\\s+"), " "
                    )
                )
            }
        )    #2
        ON CREATE SET
            mention.name = apoc.text.join(
                apoc.text.split(trim(entity.value), "\\s+"), " "
            )
        MERGE
        (page)-[s:MENTIONS_MENTION {from_model: "ned"}]->(mention)    #3

        ON CREATE SET s.start_chars= [entity.beginCharacter],
                        s.end_chars= [entity.endCharacter],
                        s.sentence_index = [entity.sentenceIndex],
                        s.type = toLower(entity.label)
        ON MATCH SET s.start_chars = s.start_chars + entity.beginCharacter,
                        s.end_chars = s.end_chars + entity.endCharacter,
                        s.sentence_index = s.sentence_index +
                        ↵entity.sentenceIndex
        WITH page, mention, entity

        FOREACH(medical in entity |
        MERGE (dis:MedicalEntity {id: medical.selected_ned_id})    #4
        ON CREATE SET dis.name=
        ↵apoc.text.join(apoc.text.split(trim(medical.selected_ned_name),
        ↵"\\s+"), " "),
                        dis.type_id = medical.selected_ned_types_id,
                        dis.types = medical.selected_ned_types,
                        dis.type = medical.selected_ned_types[0],
                        dis.original_mention = medical.value,
                        dis.definition = medical.selected_ned_definition,
                        dis.aliasess = medical.selected_ned_aliasess,
                        dis.start_chars= [entity.beginCharacter],
                        dis.end_chars= [entity.endCharacter],
                        dis.sentence_index = [entity.sentenceIndex]    #5
        ON MATCH SET dis.start_chars = dis.start_chars +
        ↵entity.beginCharacter,
                        dis.end_chars = dis.end_chars + entity.endCharacter
```

```

MERGE (mention)-[r:DISAMBIGUATED_TO]->(dis) #6
SET r.confidence = medical.selected_ned.confidence

MERGE (page)-[t:MENTIONS_ENTITY]->(dis) #7
ON CREATE SET t.sentence_index = [medical.sentenceIndex]
ON MATCH SET t.sentence_index = t.sentence_index +
    medical.sentenceIndex)

"""

self.load_in_batch(query, processed_pages, len(processed_pages),
    chunk_size=1)

```

#1 Matches node pages that are not yet processed

#2 Creates entity mention nodes and connects them to pages

#3 Adds properties, including start and end chars, to relationships connecting pages and mentions

#4 Merges medical entity nodes extracted using the scispaCy tool

#5 Adds properties, including aliases and semantic types, to the medical entity nodes

#6 Connects medical entity nodes to entity mentions

#7 Connects medical entity nodes to pages

This query stores the results of scispaCy processing in Neo4j. First it creates the `EntityMention` nodes, which are connected to the `Page` nodes. Then it creates the `MedicalEntity` nodes, enriches them with UMLS data, and connects the medical entities to `EntityMention` and the `Page` nodes. As we mentioned earlier, we kept both the `EntityMention` and the disambiguated `MedicalEntity` in the graph to improve flexibility in representing data.

7.6.4 Processing, loading, and mapping ontologies

In this phase, we load UMLS, SNOMED, and HPO ontologies into the KG. The UMLS serves as the entry point to access specific information across multiple ontologies. For this reason, we first load the SNOMED and HPO ontologies and then map each entity from them to UMLS.

INGESTING SNOMED

The following listing creates nodes and relationships in Neo4j from `sct2_Relationship_Full_US1000124_20220901.txt`.

Listing 7.14 Ingesting SNOMED: loading relationships

```
[...]
class SnomedRelationshipsImporter(BaseImporter): #1
    [...]
    def set_constraints(self): #2
        queries = [
            (
                "CREATE CONSTRAINT IF NOT EXISTS FOR (n:SnomedEntity) "
                "REQUIRE n.id IS UNIQUE"
            ),
            (
                "CREATE INDEX snomedNodeName IF NOT EXISTS "
                "FOR (n:SnomedEntity) ON (n.name)"
            ),
            (
                "CREATE INDEX snomedRelationId IF NOT EXISTS "
                "FOR ()-[r:SNOMED_RELATION]-() ON (r.id)"
            ),
            (
                "CREATE INDEX snomedRelationType IF NOT EXISTS "
                "FOR ()-[r:SNOMED_RELATION]-() ON (r.type)"
            ),
            (
                "CREATE INDEX snomedRelationUmls IF NOT EXISTS "
                "FOR ()-[r:SNOMED_RELATION]-() ON (r.uml)"
            ),
        ]

        for q in queries:
            self.connection.query(q, db=self.db)

    def import_snomed_rels(self): #3
        query = """
            UNWIND $batch as item
            MERGE (e1:SnomedEntity {id: item.sourceId})
            MERGE (e2:SnomedEntity {id: item.destinationId})
            MERGE (e1)-[:SNOMED_RELATION {id: item.typeId}]->(e2)
            FOREACH(ignoreMe IN CASE WHEN item.typeId = '116680003'
            THEN [true] ELSE [] END)
            MERGE (e1)-[:SNOMED_IS_A]->(e2) #4
        """

        size = self.get_csv_size(snomedRels_file) #5
        self.batch_store(snomed_rels_query, self.get_rows(snomedRels_file),
            size=size) #6
```

#1 Extends the BaseImporter class, which contains the base importing functionalities

#2 Defines constraints and indexes on SNOMED entities and properties

#3 Imports SNOMED relationships using a parametric query

#4 Creates the SNOMED_IS_A relationship type to track hierarchical connections between entities

#5 Gets the size of the file (default implementation in the BaseImporter class)

#6 Loads the SNOMED data in batches (implementation in the BaseImporter class)

SNOMED includes hundreds of relationships. To keep the graph schema as simple as possible, we decided to create a unique `SNOMED_RELATION`, and we store the relationship name as a `type` property. In listing 7.14 we create a `SNOMED_IS_A` relationship to define the hierarchical connec-

tions; we will see shortly why this choice is handy to propagate information from the root to the leaf nodes.

Now that we have ingested the nodes and relationships to shape the graph, we need to enrich this structure by adding names and aliases. The following class extracts information from `sct2_Description_Full-en_US1000124_20220901.txt`.

Listing 7.15 Ingesting SNOMED: loading names and aliases

```
[...]
class SnomedNamesImporter(BaseImporter):
    [...]

    def import_snomed_names(self, snomedNames_file):
        snomed_names_concepts_query = """
        UNWIND $batch as item
        MATCH (e1:SnomedEntity)
        -[r:SNOMED_RELATION {id: item.conceptId}]->
        (e2:SnomedEntity)
        WHERE item.conceptId <> '116680003' AND r.id = item.conceptId
        SET r.type = CASE
            WHEN r.type IS NULL THEN item.termAsType
            ELSE r.type END, #1
        r.aliases = CASE
            WHEN item.termAsType IN r.aliases THEN r.aliases
            ELSE coalesce(r.aliases,[]) + item.termAsType END #2
        """

        snomed_names_entities_query = """
        UNWIND $batch as item
        MATCH (e:SnomedEntity {id: item.conceptId})
        SET e.name = CASE
            WHEN e.name IS NULL THEN item.term
            ELSE e.name END, #3
        e.aliases = CASE
            WHEN item.term in e.aliases THEN e.aliases
            ELSE coalesce(e.aliases, []) + item.term END #4
        """

        size = self.get_csv_size(snomedNames_file)
        self.batch_store(
            snomed_names_concepts_query,
            self.get_rows(snomedNames_file),
            size=size)
        self.batch_store(
            snomed_names_entities_query,
            self.get_rows(snomedNames_file),
            size=size)

[...]
```

#1 Adds the relationship name in the type property

#2 Adds relationship aliases

#3 Adds the node name

#4 Adds node aliases

This class adds names and aliases to the nodes and relationships loaded into the graph. The next step involves propagating information from the root nodes to all subnodes, down to the leaf nodes. The first-level nodes represent the archetypal entities of SNOMED in the medical domain, such as diseases, body structures, substances, and events. These root nodes define the semantic types of the entities available in SNOMED. However, this information is implicit in the data we ingested because for each other

entity, there are only names and aliases in the original data. Therefore, we need a mechanism to transfer this information through the ontology's tree structure; this way, we can easily detect that a deep entity is a disease or a product. Figure 7.11 clarifies this propagation mechanism.

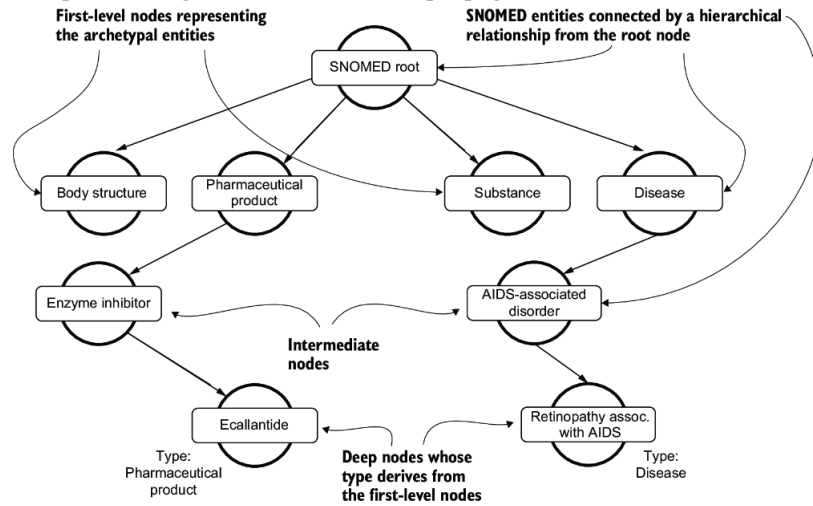


Figure 7.11 A sample of the hierarchical structure of the SNOMED ontology. Using this hierarchical structure, nodes located on a deeper level, such as *Ecallantide* and *Retinopathy associated with AIDS*, can be categorized using the information from the first-level nodes, such as *Pharmaceutical product* and *Disease*, which represent the archetypal entities of the ontology.

The next listing retrieves all nodes, following the hierarchical structure to propagate information from first-level nodes to deeper nodes.

Listing 7.16 Ingesting SNOMED: propagating labels from first-level nodes

```
[...]
class SnomedLabelPropagator():
[...]

def get_rows(self):
    propagation_query = """
        MATCH p=(n:SNOMEDEntity)-[:SNOMED_IS_A]-(m:SNOMEDEntity)
        WHERE n.id= "138875005" // Root node
        WITH distinct m as first_node

        CALL apoc.path.expandConfig(first_node, { #1
            relationshipFilter: '<SNOMED_IS_A',
            minLevel: 1,
            maxLevel: -1,
            uniqueness: 'RELATIONSHIP_GLOBAL'
        }) yield path

        UNWIND nodes(path) as other_level #2
        WITH first_node, collect(DISTINCT other_level) as uniques
        UNWIND uniques as unique_other_level
        WITH first_node, unique_other_level
        WHERE not first_node.name in coalesce(unique_other_level.type, [])

        RETURN unique_other_level.id as id, first_node.name as label - #3
    """

    with self._driver.session(database=self._database) as session:
        result = session.run(query=propagation_query)
        for record in iter(result):
            yield dict(record)
[...]
```

#1 Uses expandConfig to set the rules for traversing the hierarchy

#2 Gets the nodes resulting from this traversal process

#3 Returns all the nodes characterized by a specific type

The `SNOMED_IS_A` relationship propagates the semantic types through the tree structure by using the hierarchical connections between entities.

INGESTING HPO

The following listings outline the primary steps for ingesting the HPO ontology. More details, including the creation of the proper indices, are discussed in chapter 3 and included in the Cypher code in the book's repository. First we import the HPO ontology using the Neosemantics plugin.

Listing 7.17 Ingesting HPO: loading the ontology

```
CALL n10s.rdf.import.fetch("http://purl.obolibrary.org/obo/hp.owl", "RDF/XML");
```

The HPO ontology is available in the RDF/XML format, and it is converted into a Neo4j graph structure to be processed using Cypher.

Next, we add the HPO label to nodes representing phenotypic feature entities.

Listing 7.18 Ingesting HPO: adding the `HpoEntity` label to phenotypic features

```
MATCH (n:Resource)
WHERE n.uri STARTS WITH "http://purl.obolibrary.org/obo/HP"
SET n:HpoEntity,
    n.id = coalesce(n.id, replace(apoc.text.replace(n.uri, '(.*)obo/', ''), '_', ':'))
```

Now we need to process the annotation file that describes the connections between the phenotypic features and diseases. To make this step more efficient, we use the following query to load the disease entities as graph nodes.

Listing 7.19 Ingesting HPO: creating `HpoDiseaseEntity` nodes

```
LOAD CSV FROM 'https://github.com/obophenotype/human-phenotype-ontology/releases/latest/download/phenotype.hpoa' AS row
FIELDTERMINATOR '\t'
WITH row
SKIP 5
MERGE (dis:Resource:HpoDiseaseEntity {id: row[0]})
ON CREATE SET dis.label = row[1];
```

The next step is to create the relations between the available phenotypic features and disease nodes.

Listing 7.20 Ingesting HPO: relations between `HpoEntity` and `HpoDiseaseEntity`

```
LOAD CSV FROM 'https://github.com/obophenotype/human-phenotype-ontology/releases/latest/download/phenotype.hpoa' AS row
FIELDTERMINATOR '\t'
WITH row
SKIP 5
MATCH (dis:HpoDiseaseEntity)
WHERE dis.id = row[0]
MATCH (phe:HpoEntity)
WHERE phe.id = row[3]
MERGE (dis)-[:HAS_PHENOTYPIC_FEATURE]->(phe)
```

The following query establishes the connection between the disambiguated entities extracted from the text and the ontology nodes.

Listing 7.21 Integrating SNOMED through the UMLS

```
MATCH (m:MedicalEntity)
WITH m
MATCH (d:SnomedEntity)
WHERE m.id in d.umls_ids
WITH m, d
MERGE (m)-[:IS_SNOMED_ENTITY]->(d)
```

We perform an analogous process for the HPO ontology (see the full code for more details). The next query connects `MedicalEntity` nodes and the `HpoDiseaseEntity` nodes from the HPO annotation file.

Listing 7.22 Connecting `MedicalEntity` and `HpoDiseaseEntity` nodes

```
MATCH (m:MedicalEntity)
WITH m
MATCH (d:HpoDiseaseEntity)
WHERE m.id in d.umls_ids
WITH m, d
MERGE (m)-[:IS_DISEASE_ENTITY]->(d)
```

In the next section, we generate the co-occurrence relationship between entities located in the same sentence.

7.6.5 Generating entity co-occurrences

Identifying co-occurrences of medical entities in the text is a fundamental step in enabling advanced use cases that combine unstructured knowledge from textual content with structured knowledge in domain ontologies. The query in listing 7.23 creates a new relationship called `COOCCURR` between medical entities identified in the same sentence.

DEFINITION *Co-occurrence* refers to the projection of `Page` nodes onto `Entity` nodes.

Listing 7.23 Creating co-occurrence relationships at the sentence level

```
CALL apoc.periodic.iterate(
  "MATCH (n:Page) WHERE exists( (n)-[:MENTIONS_ENTITY]->(:MedicalEntity) )
  RETURN n",
  "MATCH (n)-[:MENTIONS_ENTITY]->(m:MedicalEntity)
  WITH n, r.sentence_index as sentences, m
  UNWIND sentences as sentence
  WITH n, sentence, collect(distinct m) as entities

  UNWIND range(0, size(entities)-2) as i
  UNWIND range(i+1, size(entities)-1) as j

  WITH n, sentence, entities, i, j
  MATCH (m1) WHERE id(m1) = id(entities[i])
  MATCH (m2) WHERE id(m2) = id(entities[j])

  WITH n, sentence, entities, i, j, m1, m2
  MERGE (m1)-[:COOCCURR]-(m2)
  ON CREATE SET s.count = 1,
    s.sentences = [sentence]
  ON MATCH SET s.count = s.count + 1,
    s.sentences = s.sentences + sentence",
  {batchSize: 50})
```

This query produces more than 25,000 relationships in the KG, making explicit the connections between medical entities located in the same sentence. As we will discuss next, analyzing the ontology connections between co-occurring entities lets us discover the established knowledge related to these entities and uncover new information that is not yet established in biomedical ontologies.

7.7 KG-based use cases

In this section, code examples will show how we handle the following use cases using KGs in combination with NED:

- Conceptual search
- Structured knowledge-based search
- KG-based interpretability and discovery
- Uncovering new knowledge

Figure 7.12 includes the analysis step related to the use cases defined in this application domain.

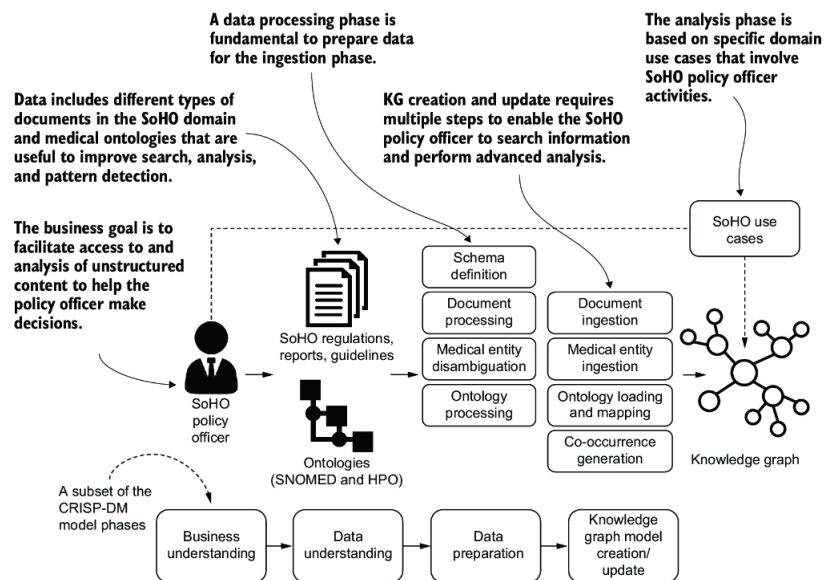


Figure 7.12 After generating the KG, we can perform analyses by applying defined use cases to address business problems and provide solutions based on the KG.

NOTE Your query results may vary slightly from those shown in the chapter due to differences in environments, Neo4j indexing behavior, data pipeline ingestion order, and the nondeterministic aspects of entity extraction performed by scispaCy.

7.7.1 Conceptual search

Conceptual search is related to the capacity to reconcile—and return to the user—different expressions with the same meaning or distinguish similar terms that refer to distinct entities. Performing a conceptual search can help us expand or focus the search for specific documents and the related portions of text that mention the entity. Figure 7.13 provides a high-level comparison of conceptual search and traditional full-text search.

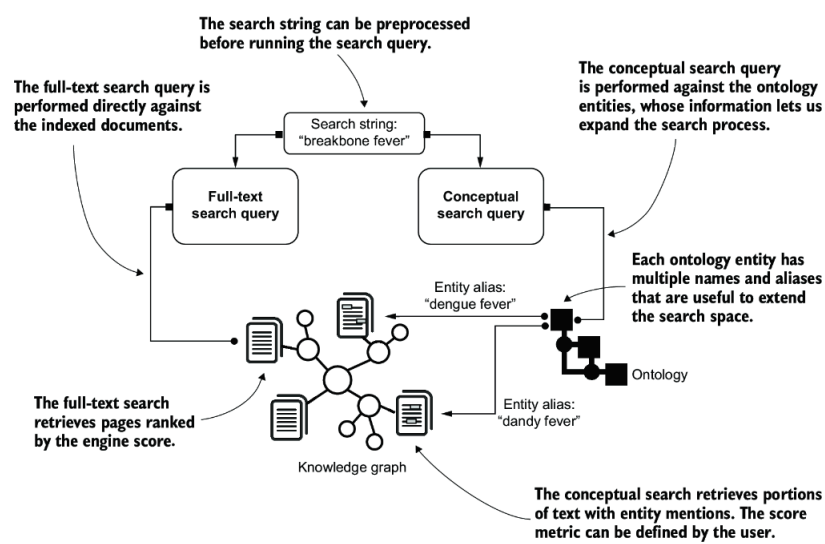


Figure 7.13 Differences between traditional full-text search and conceptual search. The full-text search query is performed directly on the set of documents, whereas the conceptual search query uses information from ontologies to expand the search space before searching the documents.

Let's examine the effect of conceptual search starting from a traditional full-text search. The following search query tries to retrieve all documents mentioning "breakbone fever."

Listing 7.24 Full-text search query with "breakbone fever" as the input string

```
CALL db.index.fulltext.queryNodes("PageText", "breakbone fever")
YIELD node, score
WITH node as p, score as score
MATCH (f:File)-[:CONTAINS_PAGE]->(p)
RETURN f.id as `File ID`, p.page_idx as `Page index`, score as Score
LIMIT 5
```

Table 7.1 shows the results of the full-text search ordered by score value. The first column lists the document path, the second column shows the page index in the document, and the third column reports the score computed by the full-text search algorithm.

Table 7.1 The top five pages retrieved by a full-text search for “breakbone fever”

File ID	Page index	Score
sample_dataset-PublicUse/ECDC Documents/west nile virus/emerging-vector-borne-diseases_annual-epidemiological-report-2014.pdf	5	2.12
sample_dataset-PublicUse/ECDC Documents/west nile virus/emerging-vector-borne-diseases_annual-epidemiological-report-2014.pdf	4	2.09
sample_dataset-PublicUse/ECDC Documents/zika virus/Communicable-disease-threats-report-26-mar-2016.pdf	10	2.08
sample_dataset-PublicUse/ECDC Documents/ebola/communicable-disease-threats-report-13-19-mar-2016.pdf	9	2.08
sample_dataset-PublicUse/ECDC Documents/zika virus/Communicable-disease-threats-report-26-mar-2016.pdf	10	2.08

This type of search identifies keywords in a set of documents by using fuzzy logic. In this case, “breakbone” is never mentioned in the text, but the term “fever” appears multiple times in the high-scored documents. Nevertheless, we are interested in identifying documents that mention this specific disease or, eventually, the causative agent of this disease.

In the UMLS knowledge base, the code C0011311 identifies this entity, which can appear in various forms, including “dengue fever,” “dungero,” and “dandy fever.” So, we next run the following query to retrieve all medical entities identified by the scispaCy model whose alias is “breakbone fever.”

Listing 7.25 Finding mentions of the entity whose alias is “breakbone fever”

```
MATCH (f:File)-[:CONTAINS_PAGE]->(p)
      -[r:MENTIONS_MENTION]->(m)-[:DISAMBIGUATED_TO]->(e)
WHERE "breakbone fever" IN [x IN e.aliases | toLower(x)] #1

UNWIND range(0, size(r.start_chars) - 1) AS mention #2
WITH f, p, e, m, r, mention
RETURN DISTINCT
  f.id AS `File ID`,
  p.page_idx AS `Page index`,
  apoc.text.join(
    collect(
      substring(
        p.text,
        apoc.coll.max([r.start_chars[mention] - 100, 0]), #3
        r.end_chars[mention] - r.start_chars[mention] + 200 #4
      )
    )[0..3], #5
    '\n\n'
  ) AS `Mention contexts`,
  size(collect(m.name)) AS `Number of mentions`
ORDER BY `Number of mentions` DESC
LIMIT 5
```

#1 Filters nodes by checking if the alias "breakbone fever" (in lowercase) exists among aliases of the entity

#2 Iterates over each mention index in the list of start characters to handle multiple mentions per page

#3 Calculates a starting index for the substring, ensuring that it doesn't go below zero by subtracting 100 from the start position

#4 Determines the length of the text snippet by computing the difference between the end and start of the mention and adding 200 for extra context

#5 Limits the collected context snippets to the first three for each file–page combination to keep the output concise

Table 7.2 shows the top-scored pages from our conceptual search and an example of a paragraph that mentions the UMLS entity representing breakbone fever, listed by the number of mentions.

Table 7.2 Top-scored pages retrieved using “breakbone fever” as an entity alias

File ID	Page index	Mention context	Number of mentions
sample_dataset-PublicUse/ECDC Documents/hepatitis-a/communicable-disease-threats-report-feb-24-2018.pdf	11	"[...] In 2017, Cambodia reported over 3,200 suspected dengue cases."	22
sample_dataset-PublicUse/ECDC Documents/ebola/communicable-disease-threats-report-17%20may-2014.pdf	11	"[...] Singapore has reported more than 1000 dengue cases nationally from January to April this year, which is 15 per cent fewer cases compared with [...]"	20
sample_dataset-PublicUse/ECDC Documents/ebola/Communicable-disease-threats-report-19-jul-2014.pdf	12	"[...] An epidemic of dengue fever in Malaysia has now infected nearly 47,000 people, which is more than double the number of cases [...]"	20
sample_dataset-PublicUse/ECDC Documents/ebola/communicable-disease-threats-report-21-jun-2014.pdf	13	"[...] Cuba has recorded 67 imported cases of dengue fever up to 8 June, according to media quoting the	20

File ID	Page index	Mention context	Number of mentions
		Cuban government.”	
sample_dataset-PublicUse/ECDC Documents/west nile virus/communicable-disease-threats-report-21-jun-2014.pdf	12	“[...] Oceania: As of 13 June 2014, 1 762 suspected dengue cases have been reported in Solomon Islands since January 2014.”	19

Comparing the results with those from the full-text search, we see that the most relevant pages are completely different. Let’s try to update the conceptual search query to show and explore the full result set (remove the `LIMIT` clause). The first result from the full-text query is located at the 19th position in the conceptual search query. This specific document has 17 mentions of the UMLS entity, whereas the top-scored result of the conceptual search query includes 22 mentions of the C0011311 UMLS entity.

Adopting conceptual search has a huge effect on our application domain. We can retrieve more precise and detailed information compared to the full-text search: in addition to the document path and the page index, we can identify the portion of text that mentions the entity and the number of occurrences of this entity on the page. Thus, conceptual search enables other features that improve the explainability of the retrieved document. Because the NED model lets us identify the position of the detected entity in the text, we can eventually perform precise debugging tests to identify the cases in which the model did not perform as expected.

Conceptual search also enhances the search process in scenarios where a word or expression has different meanings based on context. For example, the word “islands” has a traditional meaning, but it takes on a completely different semantic context when used in the expression “islands of Langerhans.” A traditional full-text search might lead to irrelevant results referring to land masses surrounded by water. By using the conceptual search mechanism, we can filter out these results and broaden the search scope to include other expressions, such as “pancreatic islets,” to capture more relevant content. As we discuss next, the core concept of conceptual search can be expanded by linking pages that include medical entities connected through ontology relationships.

7.7.2 Structured knowledge-based search

Using structured knowledge-based search, we can retrieve information from text by using formalized knowledge organized in domain ontologies and create nontrivial relationships between different pieces of text across

multiple documents. We've already seen how we can aggregate information referring to the same concept, such as dengue fever or pancreatic islets, regardless of ambiguous or multiple names. However, by using the knowledge embedded in ontologies like SNOMED, we can aggregate and connect unstructured content driven by ontology relationships. For instance, given the importance of donations of cells like the islets of Langerhans, it would be beneficial to aggregate all text that mentions diseases that can affect these cells and thus compromise the donation process. Figure 7.14 shows a mental model of the difference between a conceptual search query and a structured knowledge-based query.

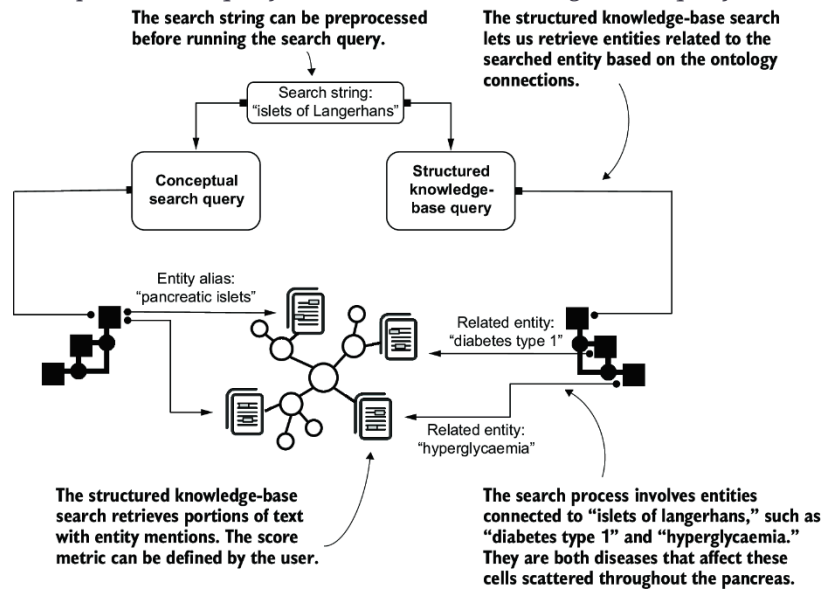


Figure 7.14 Differences between conceptual search and structured knowledge-based search. In this case, the search process involves medical entities connected by ontology relationships to the entity detected in the search query. Starting from the "islets of Langerhans" search, the system searches for "diabetes type 1" and "hyperglycaemia," which are two diseases that can affect the islets of Langerhans.

The following query retrieves portions of text that mention diseases that can affect the islets of Langerhans.

Listing 7.26 Getting text about diseases that can affect the islets of Langerhans

```
MATCH (m1:MedicalEntity)-[:IS_SNOMED_ENTITY]->(s1:SnomedEntity)
<-[r1:SNOMED_RELATION]-(s2:SnomedEntity)
<-[:IS_SNOMED_ENTITY]-(e:MedicalEntity)
WHERE m1.name = "Islets of Langerhans" AND r1.type = "FINDING_SITE" #1
WITH e
MATCH path = (f:File)-[:CONTAINS_PAGE]->(p)
-[r:MENTIONS_MENTION]->(m)-[:DISAMBIGUATED_TO]->(e)
UNWIND range(0, size(r.start_chars) - 1) AS mention #2

WITH f, p, r, e, mention,
    apoc.coll.max([r.start_chars[mention] - 100, 0]) AS start, #3
    apoc.coll.min([r.end_chars[mention] + 100, size(p.text)]) AS end #4
WITH f, p, r, e, mention, substring(p.text, start, end - start) AS context #5
WITH f, p,
    collect(DISTINCT e.name) AS `Mentioned entities`, #6
    collect(context)[0..3] AS contexts #7
RETURN DISTINCT
    f.id AS `File ID`,
    p.page_idx AS `Page index`,
    `Mentioned entities`,
    apoc.text.join(contexts, '\n\n') AS `Mention contexts`
ORDER BY size(`Mentioned entities`) DESC
LIMIT 5
```

#1 Filters for SNOMED entities related to "Islets of Langerhans" by the FINDING_SITE relationship type

#2 Iterates over each mention instance using indices from start_chars, allowing multiple mentions per relation

#3 Computes a start index for context extraction, backing up 100 characters without going below 0

#4 Calculates an end index up to 100 characters beyond the mention without exceeding the page text length

#5 Extracts a substring of the page text from the calculated start to end position as the context window

#6 Gathers a deduplicated list of medical entity names mentioned on the page

#7 Limits the collected context windows to the first three mentions

A subset of the results of this query are shown in table 7.3.

Table 7.3 Textual content about diseases that can affect the islets of Langerhans

File ID	Page index	Mentioned entities	Mention contexts
sample_dataset-PublicUse/EDQM Documents/BTC/guide-to-the-quality-and-safety-of-organs-for-transplantation-7th-edition.PDF	137	Diabetes Mellitus, Metabolic Syndrome X	"Metabolic syndrome, arterial hypertension, diabetes mellitus, albuminuria (see below) and other chronic kidney diseases or systemic disease [...]"
sample_dataset-PublicUse/EDQM Documents/BTC/guide-to-the-quality-and-safety-of-organs-for-transplantation-7th-edition.PDF	144	Hyperglycemia, Diabetes Mellitus, Non-Insulin-Dependent	" [...] Many patients with severe head trauma become hyperglycaemic and require insulin therapy, despite normal pancreatic function and no history of diabetes. [...] On the other hand, manifestation of diabetes mellitus type II is possible at an age of over 50-65 years."

In listing 7.26, we extend the logic of the conceptual search query, taking a collection of entities from the ontology as input and retrieving the related details in the documents. One of the most interesting results is from page 144 of *Guide to the Quality and Safety of Organs for Transplantation*, 7th ed., reported in column 4 of table 7.3. This page mentions *Hyperglycemia*, which is explicitly connected to the *Islets of Langerhans* entity through the `FINDING_SITE` relationship, in the context of managing a potential donor who has suffered head trauma. This sentence is located in a section dedicated to pancreas donation focused on donor-maintenance protocols. This critical information would not have been retrieved through the conceptual search mechanism. However, we used the connection of the *Hyperglycemia* medical entity to the starting point of our search (*Islets of Langerhans*) to reach this information. The other results mention different forms of diabetes mellitus, which is particularly relevant in the donation of pancreatic islets.

A structured knowledge-based search can also be performed by using the longest paths in an ontology. Let's consider the Zika virus example. This virus can be categorized as a Togavirus, and it is transmitted to humans via species of arthropods that feed on blood, especially insects and arachnids. Suppose we are interested in all diseases mentioned in the document that are caused by a Togavirus. We can explore a path that concatenates multiple `CAUSATIVE_AGENT` relationships in the SNOMED ontology, using the results to acquire the desired documents; figure 7.15 shows an example.

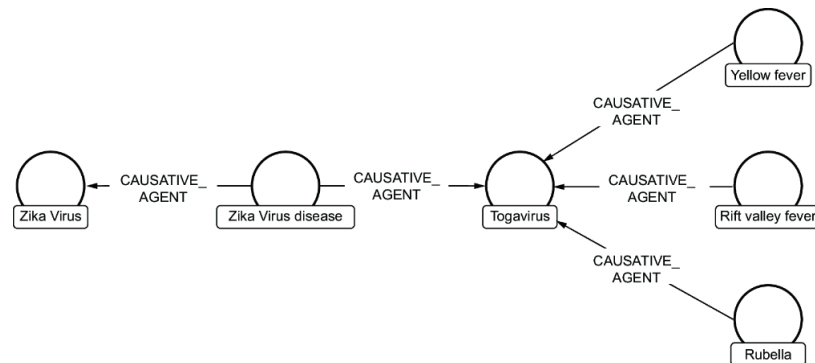


Figure 7.15 SNOMED ontology paths including only the `CAUSATIVE_AGENT` relationship. Navigating this subset of relationships is particularly useful for identifying diseases carried by the same virus types.

We can expand the search to involve entities such as *Yellow fever*, *Rift valley fever*, and *Rubella* that do not have a direct connection with the entity specified in the search query (*Zika virus*). Listing 7.27 shows the query to use these complex paths for retrieving relevant documents. This example is particularly exciting for our application domain because we can retrieve documents mentioning similar diseases based on the type of virus they have in common.

Listing 7.27 Getting documents mentioning diseases caused by Togaviruses

```

MATCH (m1:MedicalEntity)-[:IS_SNOMED_ENTITY]->(s1:SnomedEntity)
-[r1:SNOMED_RELATION*3..3]-(s2:SnomedEntity)
<-[:IS_SNOMED_ENTITY]-(e:MedicalEntity) #1
WHERE m1.name = "Zika Virus"

AND all(x IN r1 WHERE x.type = "CAUSATIVE_AGENT") #2
WITH DISTINCT e

MATCH path = (f:File)-[:CONTAINS_PAGE]->(p)
-[r:MENTIONS_MENTION]->(m)-[:DISAMBIGUATED_TO]->(e)
WITH f, e, collect(p.page_idx) AS pages_list #3
RETURN DISTINCT
  f.id AS `File ID`,
  pages_list,
  collect(DISTINCT e.name) AS `Mentioned entity`
ORDER BY size(`Mentioned entity`) DESC
LIMIT 5
  
```

#1 Traverses three `SNOMED_RELATION` hops to discover indirectly related medical entities

#2 Filters paths where all three relationships are `CAUSATIVE_AGENT`, to ensure semantic consistency

#3 Collects indices from all pages in which the entity is mentioned

The results of this query are listed in table 7.4.

Table 7.4 Documents and pages mentioning diseases caused by Togaviruses

File ID	Pages	Mentioned entities
sample_dataset-PublicUse/ECDC Documents/west nile virus/TER-Mosquito-surveillance-guidelines.pdf	10	<i>Murray valley encephalitis, Japanese Encephalitis, Venezuelan equine encephalomyelitis, Encephalomyelitis, Eastern Equine, Ross river virus infection</i>
sample_dataset-PublicUse/ECDC Documents/ebola/ebola-preparedness-belgium.pdf	38	<i>Yellow Fever, Rift Valley Fever, West Nile Fever, Dengue Fever, Chikungunya Fever</i>
sample_dataset-PublicUse/ECDC Documents/west nile virus/communicable-disease-threats-report-18-august-2019-updated-26-august-2019.pdf	1	<i>Rubella, Yellow Fever, Japanese Encephalitis, Dengue Fever, Chikungunya Fever</i>
sample_dataset-PublicUse/ECDC Documents/ebola/communicable-disease-threats-report-18-august-2019-updated-26-august-2019.pdf	1	<i>Rubella, Yellow Fever, Japanese Encephalitis, Dengue Fever, Chikungunya Fever</i>
sample_dataset-PublicUse/ECDC Documents/west nile virus/communicable-disease-threats-report-15-december-2018.pdf	1	<i>Yellow Fever, Dengue Fever, Mosquito-Borne Diseases, Chikungunya Fever</i>

This use case is particularly useful for automatically retrieving relevant information using ontology connections. Starting from the entity mentioned in the search string, we can navigate between multiple pieces of text that mention entities related to this search entity, opening new possibilities to enrich the user experience in an IAS system.

7.7.3 KG-based interpretability and discovery

Starting from the co-occurrence of entities in the same sentence, KG-based interpretability and discovery lets us analyze how these entities are connected in the ontologies and what the nature of this connection is. In some cases, the ontology connections reflect the reason for the co-occurring entities in a specific sentence (interpretability). This analysis can also validate co-occurring entities against an ontology. And other cases,

ontology connections add knowledge that expands the information provided by the sentence (discovery).

To better understand the value of this use case, consider listing 7.28, which includes sentences from the document entitled “Proposal for a Directive of the European Parliament and of the Council Setting Standards of Quality and Safety for the Collection, Testing, Processing, Storage, and Distribution of Human Blood and Blood Components and Amending Council Directive 89/381/EEC.” In this document, “AIDS” and “Hepatitis” co-occur two times.

Listing 7.28 Sentences where *AIDS* and *Hepatitis* co-occur

```
SENTENCE 1: The reasons why they should not donate which put recipients at risk, such as unsafe sexual behaviour, HIV/ AIDS, hepatitis, drug addiction and the use and abuse of drugs;
```

```
[...]
```

```
SENTENCE 2: Infectious diseases persons suffering or having suffered from  
- Babesiosis  
- Hepatitis B (HBsAg confirmed positive)  
- Hepatitis C  
- Hepatitis, infectious (of unexplained aetiology)  
- HIV/AIDS
```

“AIDS” and “Hepatitis” appear close to each other in these sentences because they both represent a risk factor in the donation process: they are infectious diseases. This information is directly encoded in SNOMED, and expressed by different ontology paths. The following listing shows a subset.

Listing 7.29 Paths connecting entities, interpretability perspective

```
(AIDS)-[:PATHOLOGICAL_PROCESS]->(Infectious disease)  
<-[:DUE_TO]-(Hepatitis due to infection)  
-[:IS_A]->(Inflammatory disorder of liver)  
  
(AIDS)-[:PATHOLOGICAL_PROCESS]->(Infectious disease)  
<-[:DUE_TO]-(Viral hepatitis)  
-[:IS_A]->(Inflammatory disorder of liver)
```

These ontology paths define connections between these two entities and encode the meaning of the co-occurrence of “AIDS” and “Hepatitis” (which is labeled “Inflammatory disorder of liver” in the SNOMED ontology). We also validate the extracted entities against the ontology, which enables us to understand why these entities were extracted in the same sentence.

Other types of SNOMED paths reveal interesting connections between AIDS and hepatitis. For example, the first SNOMED path in the next listing shows that AIDS is related to a disorder defined as “Hepatomegaly associated with AIDS” that affects the liver. The second entry reports that a specific form of hepatitis called “Lupus hepatitis” involves the immune system as “AIDS.”

Listing 7.30 Paths connecting entities, discovery perspective

```
(AIDS)-[:ASSOCIATED_WITH]-(Hepatomegaly associated with AIDS (disorder))
-[:FINDING_SITE]->(Liver)
<-[:FINDING_SITE]-(Inflammatory disorder of liver)

(AIDS)-[:HAS_DEFINITIONAL_MANIFESTATION]->(Immune system finding)
<-[:HAS_DEFINITIONAL_MANIFESTATION]-(Lupus hepatitis)
-[:IS_A]->(Inflammatory disorder of liver)
```

The ontology supports us in understanding why a pair of medical entities appear in the sentence and lets us discover new details related to this pair. Figure 7.16 illustrates integrating domain ontologies into a KG built from natural language content; it shows “Dengue” and “Zika virus” as examples of co-occurrence.

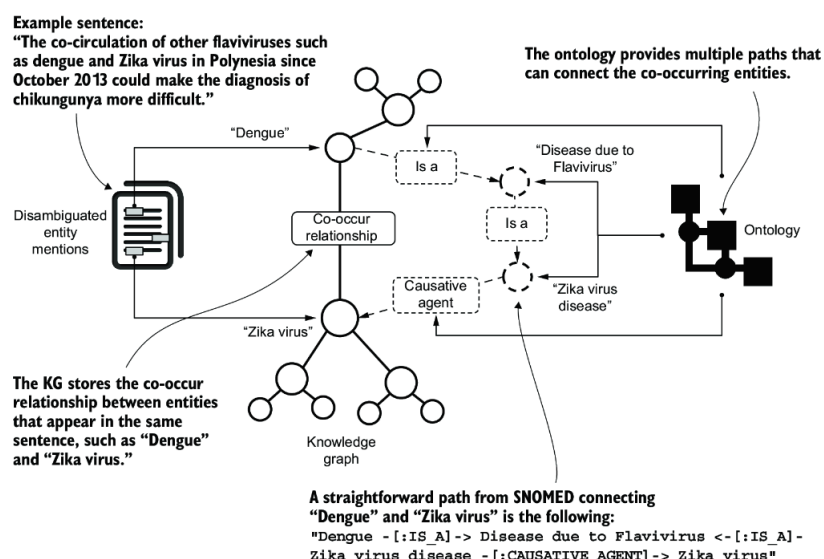


Figure 7.16 Mental model describing the KG-based interpretability and discovery process. Co-occurring entities in the text can be connected by multiple paths in the SNOMED ontology.

Let’s dive into the details and see how to apply practical solutions in the SoHO domain to enable interpretability and discovery with a real KG. The following listing retrieves the top five entity types that co-occur with “Zika virus.”

Listing 7.31 Retrieving the top entity types co-occurring with “Zika virus”

```
MATCH (m1:MedicalEntity)-[r:COOCCURR]-(m2:MedicalEntity)
WHERE m1.id= "C0318793"
RETURN m2.type as `Entity Type`, count(m2.type) as `Number of co-occurrences`
ORDER BY count(m2.type) DESC
LIMIT 5
```

This query computes the number of co-occurring entity types to identify the most relevant ones. The results are shown in table 7.5.

Table 7.5 Top entity types co-occurring with “Zika virus”

Entity	Number of co-occurrences
Geographic Area	255
Qualitative Concept	132
Disease or Syndrome	125
Functional Concept	106
Finding	98

Most of the results include `Geographic Area`. This is related to the fact that “Zika virus” is often mentioned in bulletins that report the spread of communicable diseases. Other categories, such as `Qualitative Concept` and `Functional Concept`, include broader concepts that in most cases are not relevant for our example. For this reason, we will focus on the co-occurring `Disease or Syndrome` entities. The next query finds co-occurring entities and the portion of text from which this co-occurrence was extracted.

Listing 7.32 Getting co-occurring disease entities with context

```

MATCH (n1:MedicalEntity)-[r:COOCCURR]-(n2:MedicalEntity)
WHERE n1.id = "C0318793" AND n2.type = "Disease or Syndrome" #1
WITH n1, r, n2
ORDER BY r.count DESC
MATCH (f:File)-[:CONTAINS_PAGE]->(p:Page)
      -[r1:MENTIONS_MENTION]->(m1)-[:DISAMBIGUATED_TO]->(n1),
      (p)-[r2:MENTIONS_MENTION]->(m2)-[:DISAMBIGUATED_TO]->(n2)
WHERE r1.sentence_index = r2.sentence_index #2
WITH f, p, r1, r2, n2
RETURN DISTINCT
  f.id AS `File ID`,
  p.page_idx AS `Page index`,
  n2.id AS `Co-occurring entity id`,
  n2.name AS `Co-occurring entity name`,
  CASE
    WHEN r1.end_chars[0] > r2.start_chars[0]
    THEN substring(p.text, r2.start_chars[0],
      ↵r1.end_chars[0] - r2.start_chars[0]) #3
    ELSE substring(p.text, r1.start_chars[0],
      ↵r2.end_chars[0] - r1.start_chars[0]) #4
  END AS `Mentions context`

```

#1 Filters for entities co-occurring with "C0318793" (Zika virus) and restricts to those of type "Disease or Syndrome"

#2 Ensures that both entities were mentioned in the same sentence on the page

#3 Captures the co-occurrence span

#4 Ensures that the context window always runs from the earlier to the later character position

Table 7.6 reports a subset of the results; you can analyze the full result set by running the query in listing 7.32.

Table 7.6 Disease entities that co-occur with “Zika virus” and context

File ID	Page index	Co-occurring entity ID	Co-occurring entity name	Mentions context
sample_dataset-PublicUse/ECDC Documents/zika virus/Communicable-disease-threats-report-23-apr-2016.pdf	8	C0009450	Communicable Diseases	“COMMUNICABLE DISEASE THREATS REPORT Countries and territories with reported confirmed autochthonous cases of Zika virus”
sample_dataset-PublicUse/ECDC Documents/ebola/Communicable-disease-threats-report-01-jul-2017.pdf	17	C0009450	Communicable Diseases	“COMMUNICABLE DISEASE THREATS REPORT Zika: No mosquito-borne Zika virus”
sample_dataset-PublicUse/ECDC Documents/zika virus/Questions and answers about Zika virus disease.pdf	4	C0276289	Zika Virus Infection	“Zika virus disease Risk of transmission in Europe What is the risk to Europe? The main risk to the EU is related to an infected traveller introducing the virus to an area with mosquitoes that are able to transmit Zika virus”
sample_dataset-PublicUse/ECDC Documents/west nile virus/Communicable-disease-threats-report-2-jul-2016.pdf	1	C0018378	Guillain-Barre Syndrome	“Zika virus is a cause of microcephaly and Guillain-Barre syndrome”
sample_dataset-PublicUse/EDQM Documents/BTC/optimal-use-of-clotting-factors-and-platelets-european-symposium-	300	C0011311, C0008055	<i>Dengue Fever, Chikungunya Fever</i>	“chikungunya and dengue virus infections in France and overseas departements, and emerging

File ID	Page index	Co-occurring entity ID	Co-occurring entity name	Mentions context
proceedings-wildbad-kreuth-initiative-iv-freising-germany.PDF				pathogens such as Zika virus”

The entities *Zika virus* and *Communicable Diseases* (UMLS ID C0009450) occur many times in the same sentence. This is because many of the ingested bulletins have a similar structure, such as “COMMUNICABLE DISEASE THREATS REPORT Zika: No mosquito-borne Zika virus [...]”. However, other co-occurrence examples are more relevant for interpretability and discovery. For instance, *Zika virus* often co-occurs with the following entities: *Zika Virus Infection* (UMLS ID C0276289), *Dengue Fever* (UMLS ID C0011311), and *Chikungunya Fever* (UMLS ID C0008055).

The co-occurrence with *Zika Virus Infection* is understandable. However, the semantic connection between these two entities is directly available in the SNOMED ontology. The following query uses the corresponding SNOMED IDs and labels to show the SNOMED connection between *Zika virus disease* (SNOMED ID 3928002) and *Zika virus* (SNOMED ID 50471002).

Listing 7.33 Getting paths between *Zika virus disease* and *Zika virus*

```

MATCH (s1), (s2)
WHERE s1.id = "3928002" AND s2.id = "50471002"
WITH s1, s2, allShortestPaths((s1)-[:SNOMED_RELATION*1..8]-(s2)) AS paths #1
UNWIND paths AS path
WITH relationships(path) AS path_edges,
     nodes(path) AS path_nodes
WITH
  [n IN path_nodes | n.name] AS node_names,
  [r IN path_edges | COALESCE(r.type, 'IS_A')] AS rel_types, #2
  [n IN path_edges | startnode(n).name] AS rel_starts #3
WITH [i IN range(0, size(node_names) - 1) |
     CASE
       WHEN i = size(node_names) - 1
       THEN '(' + node_names[i] + ')' #4
       WHEN node_names[i] = rel_starts[i]
       THEN '(' + node_names[i] + ')' + '-[:' + rel_types[i] + ']->' #5
       ELSE '(' + node_names[i] + ')' + '<-[:' + rel_types[i] + ']-' #6
     END
] AS string_paths
RETURN DISTINCT apoc.text.join(string_paths, '') AS `Extracted paths`

```

#1 Finds shortest paths (up to eight hops) between two SNOMED concepts using SNOMED_RELATION

#2 Uses the relationship type (default "IS_A") to ensure valid labeling

#3 Retrieves the name of the starting node for each relationship to determine the relationship direction

#4 Completes the path string

#5 Indicates that the relationship is outgoing from the current node

#6 Indicates that the relationship is incoming to the current node

This query returns all the shortest paths between *Zika virus* and *Zika virus disease* in a pretty-printed format. The result is shown next.

Listing 7.34 Paths connecting *Zika virus disease* and *Zika virus*

```
(Zika virus disease)-[:CAUSATIVE_AGENT]->(Zika virus)
```

In this case, the SNOMED ontology provides a relationship that clearly “interprets” the meaning of this co-occurrence. Such a connection reflects the linguistic and conceptual proximity of the disease and its causative agent in clinical and epidemiological texts. In our scenario, we provide graph-based augmented information, which makes explicit why these co-occurring entities can be validated.

As reported in table 7.6, the *Zika virus* medical entity also co-occurs with another disease, *Dengue Fever*. We can explore the ontology paths connecting these two entities by running the query in listing 7.33 and replacing the `WHERE` clause with `s1.id="38362002" AND s2.id="50471002"`. The results are as follows.

Listing 7.35 Paths connecting *Dengue* and *Zika virus*

```
(Dengue)-[:IS_A]->(Disease due to Flavivirus)
<-[:IS_A]-(Zika virus disease)-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:IS_A]->(Mosquito-borne flavivirus fever)
<-[:IS_A]-(Zika virus disease)-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:IS_A]->(Viral disease)
<-[:IS_A]-(Zika virus disease)-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(Infectious process
↔(qualifier value))
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Congenital Zika virus infection)
-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:CAUSATIVE_AGENT]->(Arthropod-borne organism)
<-[:CAUSATIVE_AGENT]-(Zika virus disease)
-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(Infectious process
↔(qualifier value))
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Zika virus disease)
-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:PATHOLOGICAL_PROCESS]->(Infectious disease)
<-[:PATHOLOGICAL_PROCESS]-(Zika virus disease)
-[:CAUSATIVE_AGENT]->(Zika virus)

(Dengue)-[:CAUSATIVE_AGENT]->(Togavirus)
<-[:CAUSATIVE_AGENT]-(Zika virus disease)
-[:CAUSATIVE_AGENT]->(Zika virus)
```

Multiple paths connect *Dengue* and *Zika virus*, including key entities such as *Viral disease* and *Infectious disease* that allow us to interpret why these diseases co-occur in the same sentence (see the last row of table 7.6). In addition, entities such as *Disease due to Flavivirus*, *Togavirus*, and *Mosquito-borne flavivirus fever* provide insights into how *Dengue* and *Zika virus* are connected, regardless of the sentence in which they are mentioned.

This validation process can be extended to other biomedical ontologies, including the HPO. For example, the results of the following query demonstrate how co-occurrence patterns between phenotypic features (e.g., *Renal cell carcinoma*, *Leukemia*) and associated disease entities (e.g., *von Hippel-Lindau syndrome*, *Colorectal cancer*, *RETINOBLASTOMA*) can be systematically extracted and validated. This approach enables the identification of clinically relevant phenotype–disease associations grounded in document-level evidence.

Listing 7.36 Finding HPO disease entities that co-occur with phenotypic features

```
MATCH (d:HpoDiseaseEntity)<-[:IS_DISEASE_ENTITY]-(n1:MedicalEntity)
      -[r:COOCCURR]-(n2:MedicalEntity)-[:IS_HPO_ENTITY]->(h:HpoEntity)
WHERE EXISTS((d)-[:HAS_PHENOTYPIC_FEATURE]->(h))
      AND n2.id in ["C0007134", "C0376545"] #1
WITH n1, n2, d, h

MATCH (f:File)-[:CONTAINS_PAGE]->(p:Page)
      -[r1:MENTIONS_MENTION]->(m1)-[:DISAMBIGUATED_TO]->(n1),
      (p)-[r2:MENTIONS_MENTION]->(m2)-[:DISAMBIGUATED_TO]->(n2)
WHERE r1.sentence_index = r2.sentence_index #2
WITH f, p, r1, r2, n1, n2, d, h

RETURN DISTINCT
  f.id AS `File name`,
  p.page_idx AS `Page index`,
  h.label AS `HpoEntity name`,
  d.label AS `HpoDiseaseEntity name`,
  CASE
    WHEN r1.end_chars[0] > r2.start_chars[0]
    THEN substring(p.text, r2.start_chars[0], r1.end_chars[0]
      - r2.start_chars[0]) #3
    ELSE substring(p.text, r1.start_chars[0], r2.end_chars[0]
      - r1.start_chars[0]) #4
  END AS `Mentions context`
```

#1 Selects pairs of medical entities where n1 co-occurs with a fixed subset of phenotypic entities and is linked to a disease d that has the phenotype h

#2 Filters the results to mentions that appear in the same sentence

#3 Extracts the surrounding text to form a co-occurrence snippet

#4 Ensures the correct ordering for text extraction

Unlike earlier examples, here we specify that one medical entity must be a disease and the other a phenotypic feature indexed in the HPO annotation file. The results of this query are listed in table 7.7.

Table 7.7 HPO disease entities co-occurring with phenotypic features

File ID	Page index	HPO name	HpoDisease name	Mentions context
sample_dataset-PublicUse/EDQM Documents/BTC/guide-to-the-quality-and-safety-of- organs-for- transplantation-7th-edition.PDF	275	Renal cell carcinoma	von Hippel-Lindau syndrome	"[...] von Hippel-Lindau syndrome could be considered (preferably the heart donation) when inappropriate risks associated with RCC [...]"
sample_dataset-PublicUse/EDQM Documents/BTC/guide-to-the-quality-and-safety-of- organs-for- transplantation-7th-edition.PDF	228	Renal cell carcinoma	Colorectal cancer	"[...] "RCCs, t neuro-endocrine carcinomas one breast cancer, one colorectal cancer [...]"
sample_dataset-PublicUse/BTC key docs/CELEX_52002PC0319_EN_TXT.pdf	12	Leukemia	RETINOBLASTOMA	"[...] retinoblastoma melanoma of the anterior pole, haematological neoplasm [...]"

This use case has practical applications for analysts, who can derive interesting patterns from co-occurring entity types. For example, in the case of multiple co-occurring diseases, we can verify which ontology paths connect these diseases. On the other hand, the absence of interesting paths between co-occurring entities signals new knowledge that is not encoded in biomedical ontologies; we discuss this case next.

7.7.4 Uncovering new knowledge

In a context that changes quickly due to research advancements, some knowledge in the text has not yet been consolidated in medical ontologies. As a consequence, entities that co-occur in the same sentence may not have a direct match in terms of interpretability and discovery in the domain ontologies.

Instead of using the ontology to enrich the information of the KG, we can use a KG built from textual content to suggest new facts that can be integrated into the ontology. This use case allows us to identify new patterns in the text that can be useful in the medical domain. To identify this type

of pattern, let's recap the diseases that co-occur most with *Zika virus* by running the following query.

Listing 7.37 Finding the top diseases co-occurring with “Zika virus” in text

```
MATCH (n1:MedicalEntity)-[r:COOCCURR]-(n2:MedicalEntity)
WHERE n1.id= "C0318793" and n2.type = "Disease or Syndrome"
RETURN n2.name, r.count
ORDER BY r.count DESC
LIMIT 5
```

The results are shown in table 7.8.

Table 7.8 Top diseases co-occurring with *Zika virus*

Disease entity	Occurrences
Communicable Diseases	606
Zika Virus Infection	520
Dengue Fever	206
Guillain-Barre Syndrome	195
Chikungunya Fever	155

In the results, several diseases are associated with the *Zika virus* entity. Among them, Guillain-Barre Syndrome stands out because, unlike the others, it is not a vector-borne disease. This makes its co-mention potentially unexpected and clinically significant, suggesting a possible complication rather than mere co-endemicity.

For this reason, it is helpful to get further details by exploring the textual content that mentions both Zika virus and Guillain-Barre syndrome. We can reuse the query in listing 7.32 by replacing the first `WHERE` clause with `n1.id= "C0318793" AND n2.id = "C0018378"`, which are the UMLS ids of Zika virus and Guillain-Barre syndrome, respectively (we also removed from the results the “Co-occurring entity id” and the “Co-occurring entity name” fields because they are not useful in this case). The cleaned results of this query, obtained by removing redundant results, are shown in table 7.9.

Table 7.9 Text mentioning both Zika virus and Guillain-Barre syndrome

File ID	Page index	Mentions context
sample_dataset-PublicUse/ECDC Documents/ebola/communicable-disease-threats-report--20-feb-2016.pdf	3	"Zika and potential complications published on 12 February 2016, six countries have reported an increase in the incidence of cases of microcephaly (Brazil) and Guillain-Barre syndrome"
sample_dataset-PublicUse/ECDC Documents/zika virus/zika-virus-rapid-risk-assessment-11-april-2016.docx.pdf	2	"Zika virus epidemic, health services and practitioners should be alerted to the possible occurrence of neurological syndromes (Guillain-Barre syndrome)"
sample_dataset-PublicUse/ECDC Documents/ebola/communicable-disease-threats-report-16-june-2019.pdf	3	"Guillain-Barre is known to be triggered by bacterial infections, respiratory viruses, entero- viruses, and arboviruses such as dengue and Zika"

The results state that Zika virus is one of the causes of Guillain-Barre syndrome. So, we expect ontologies such as SNOMED to include a fact such as `Guillain-Barre syndrome-[ :CAUSATIVE_AGENT ]->Zika virus`. Let's check by running the query in listing 7.33 and replacing *Zika virus disease* (SNOMED ID 3928002) with *Infectious neuronitis* (the SNOMED term for Guillain-Barre syndrome; SNOMED ID 40956001) in the `WHERE` clause. The following listing shows a subset of the results: multiple SNOMED ontology paths connect these entities, but most of these paths do not represent meaningful connections between.

Listing 7.38 Paths connecting *Zika virus* and *Infectious neuronitis*

```
(Zika virus)<-[:CAUSATIVE_AGENT]-(Congenital Zika virus infection)
-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(Infectious process (qualifier value))
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Reactivation of hepatitis B viral hepatitis)
-[:ASSOCIATED_MORPHOLOGY]->(Inflammation)
<-[:ASSOCIATED_MORPHOLOGY]-(Infectious neuronitis)

(Zika virus)<-[:CAUSATIVE_AGENT]-(Zika virus disease)
-[:CAUSATIVE_AGENT]->(Togavirus)
<-[:CAUSATIVE_AGENT]-(Mosquito-borne flavivirus encephalitis)
-[:ASSOCIATED_MORPHOLOGY]->(Inflammation)
<-[:ASSOCIATED_MORPHOLOGY]-(Infectious neuronitis)
```

In the first entry, *Zika virus* is connected to *Infectious neuronitis* because Zika virus is the causative agent of *Congenital Zika virus infection*, which is an infectious process. The issue is that the *Infectious process* (qualifier value) node is connected to multiple contagious processes, such as *Reactivation of hepatitis B viral hepatitis* (disorder). This disorder is associated with inflammation, which is also a morphological manifestation of *Infectious neuronitis*. Similar reasoning can be applied to the second entry in listing 7.38. These paths were retrieved because they include hub nodes, such as *Infectious process* (qualifier value) and *Inflammation*, which are connected to a massive number of nodes related to infectious processes.

One solution is to filter out paths that include these hub nodes. To do so, we can use graph data science algorithms provided by the Neo4j Graph Data Science Library (GDS), which offers tools to compute the centrality of a node in the graph. To run this algorithm, the first step is to create a graph projection that includes the relevant set of nodes and relationships to perform our analysis. In this case, we want to focus on the SNOMED ontology graph, and we can run the following code.

Listing 7.39 Creating a projection on the SNOMED ontology

```
CALL gds.graph.project(
  'snomedGraph',
  'SnomedEntity',
  'SNOMED_RELATION'
)
YIELD
  graphName AS graph,
  relationshipProjection AS knowsProjection,
  nodeCount AS nodes,
  relationshipCount AS rels;
```

After generating the projection, we can run the degree algorithm in streaming mode to retrieve the most-connected nodes and use this result to filter out paths that are not relevant for our purpose.

Listing 7.40 Filtering out hub nodes

```
CALL gds.degree.stream('snomedGraph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId).name AS name, score AS degree
ORDER BY degree DESC
LIMIT 350
WITH collect(name) AS hub_nodes #1

MATCH (s1), (s2)
WHERE s1.id = "3928002" AND s2.id = "40956001"
WITH s1,
      s2,
      allShortestPaths((s1)-[:SNOMED_RELATION*1..8]-(s2)) AS paths,
      hub_nodes #2
UNWIND paths AS path
WITH relationships(path) AS path_edges,
nodes(path) AS path_nodes,
hub_nodes
WITH
[n IN path_nodes | n.name] AS node_names,
[r IN path_edges | COALESCE(r.type, 'IS_A')] AS rel_types, #3
[n IN path_edges | startnode(n).name] AS rel_starts, #4
hub_nodes
WHERE NOT any(x IN node_names WHERE x IN hub_nodes) #5
WITH [i IN range(0, size(node_names) - 1) |
CASE
WHEN i = size(node_names) - 1
THEN '(' + node_names[i] + ')' #6
WHEN node_names[i] = rel_starts[i]
THEN '(' + node_names[i] + ')' + '-[:' + rel_types[i] + ']->' #7
ELSE '(' + node_names[i] + ')' + '<-[:' + rel_types[i] + ']-' #8
END
] AS string_paths
RETURN DISTINCT apoc.text.join(string_paths, '') AS `Extracted paths`
```

#1 Collects the top 350 high-degree nodes (central hubs)

#2 Finds all shortest paths (1-8 hops) between the two SNOMED concept nodes

#3 Defaults the relationship type to "IS_A" if it's missing

#4 Retrieves the name of the source node for each relationship to determine directionality in the final string

#5 Excludes any path that passes through a hub node

#6 Marks the end of the path

#7 Renders a forward (outgoing) relationship between the current node and the next

#8 Renders a backward (incoming) relationship from the next node to the current one

This query combines the nodes' degree computation with the detection of relevant paths connecting two entities. Paths that include nodes with a higher degree are excluded from the results. The next listing shows the results after this filtering process.

Listing 7.41 Revised paths connecting *Zika virus disease* and *Infectious neuronitis*

```
(Zika virus disease)-[:IS_A]->(Viral disease)
<-[:IS_A]-(Erythema multiforme caused by virus (disorder))
-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(AI – autoimmune)
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Infectious neuronitis)

(Zika virus disease)-[:IS_A]->(Disease due to Flavivirus)
<-[:IS_A]-(Acute hepatitis C)
-[:IS_A]->(Acute inflammatory disorder)
<-[:IS_A]-(Infectious neuronitis)

(Zika virus disease)<-[:ASSOCIATED_FINDING]-(Presumptive recent Zika negative)
-[:IS_A]->(Context-dependent finding (context-dependent category))
<-[:IS_A]-(History of Guillain Barre syndrome (situation))
-[:ASSOCIATED_FINDING]->(Infectious neuronitis)

(Zika virus disease)-[:IS_A]->(Viral disease)
<-[:IS_A]-(Common cold)-[:IS_A]->(Acute inflammatory disorder)
<-[:IS_A]-(Infectious neuronitis)

(Zika virus disease)-[:IS_A]->(Viral disease)
<-[:IS_A]-(VHF – Viral haemorrhagic fever)
-[:IS_A]->(Acute inflammatory disorder)
<-[:IS_A]-(Infectious neuronitis)

(Zika virus disease)<-[:ASSOCIATED_FINDING]-(Presumptive recent Zika positive)
-[:IS_A]->(Context-dependent finding (context-dependent category))
<-[:IS_A]-(History of Guillain Barre syndrome (situation))
-[:ASSOCIATED_FINDING]->(Infectious neuronitis)

(Zika virus disease)-[:IS_A]->(Viral disease)
<-[:TEMPORALLY_FOLLOWS]-(Transient paroxysmal cold hemoglobinuria)
-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(AI – autoimmune)
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Infectious neuronitis)

(Zika virus disease)-[:IS_A]->(Viral disease)
<-[:DUE_TO]-(Erythema multiforme caused by virus (disorder))
-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(AI – autoimmune)
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Infectious neuronitis)

(Zika virus disease)-[:IS_A]->(Viral disease)
<-[:AFTER]-(Transient paroxysmal cold hemoglobinuria)
-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]->(AI – autoimmune)
<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Infectious neuronitis)
```

This time, we retrieved a result set of 9 entries, compared to the 11,185 paths detected with the base query. However, the results still include general entities such as *Viral disease*, which connects *Zika virus disease* to random viral diseases that can be used to connect *Zika virus disease* to *Infectious neuronitis*. In other words, a direct connection between these entities is not encoded in the SNOMED ontology. This is a typical example in which the co-occurrence of entities defined in the KG can potentially be used to enrich a domain ontology in a virtuous circle.

This mechanism can be exponentially enhanced by extracting meaningful relationships from the text. In this approach, the starting point is represented by the extracted relationships, which are more specific than the co-occurrence connection; see figure 7.17.

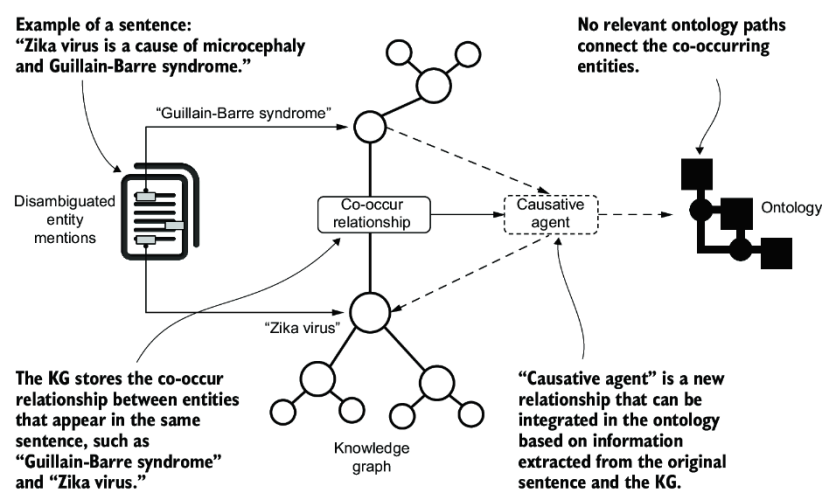


Figure 7.17 Mental model describing the process of uncovering new knowledge. In this case, "Zika virus" and "Guillain-Barre syndrome" appear multiple times in the same sentences, but the medical ontologies do not provide any relevant connections between these entities.

Summary

- Named entity disambiguation (NED) lets us connect entities mentioned in the text to a reference knowledge base.
- Combining NED with KG technologies opens new opportunities to develop advanced services in critical domains.
- Constructing a KG requires multiple steps, including defining the schema, ingesting documents, disambiguating entities from these documents, integrating domain ontologies, mapping extracted entities, and generating co-occurrence relationships between entities located in the same sentence.
- Advanced analysis can be performed on the KG to support different use cases in the application domain: conceptual search, structured knowledge-based search, KG-based interpretability and discovery, and uncovering new knowledge.